Agent 开发面经

基于“轻记”工程的逐题回答

覆盖原题 23 轮 / 642 题

更新日期：2026-10-27（上海）

依据当前工作区源码、文档与既有验收记录；未读取个人健康数据库或密钥。

使用约定

每题保留原公司、轮次和题号。回答优先描述已实现行为；“设计/扩展/未实现”表示推演能力，不是项目经历；个人职级、团队、客户与贡献无法从源码确认，需据实补充。相同问题在不同场次重复出现时保留回答，便于按轮模拟面试。

代码路径均相对于工程根目录。既有 JSON 报告中的测试/性能数字是历史记录，本次没有重跑后端全套或真实模型评测；文末编码练习已在隔离脚本中运行。工程中的个人数据、凭据和隐私原文不进入本材料。

原始 PDF 保留，当前文件为新增问答版；附带 Markdown 源稿，方便继续按实际经历修改。

先记住项目边界

已经实现

Taro 微信小程序、FastAPI API、固定 LangGraph 图、SQLite 事件与检查点；持久化追问和计划确认；四层记忆与结构化冲突处理；最小模型外发、受控输出；隐私删除血缘、后台任务租约与 outbox；本地遥测与合成回归。

尚未实现

多用户/多租户鉴权、开放 ReAct、动态画布 DSL、通用 Skill/MCP、文档向量 RAG、任意代码/Browser Use、多 Agent、SSE token 续传、独立 Engine 服务和多实例扩容。

可以引用的数字

默认 2 个活跃工作流、后台并发 1；执行期限 120 秒、模型超时 45 秒、安全分类超时 8 秒；请求 64 KiB、模型响应 1 MiB。配置上限不是吞吐容量。安全报告：16 条合成样本，10 条风险均拦截，6 条普通均放行，2 条风险路由与标注不同。

最值得讲的技术线

一条记录从草稿到追问再到保存；一次恢复从 request/thread/interrupt 到回执去重；一次删除如何沿血缘清理并阻止旧结果写回。每条线都能连接到真实函数与测试。

三条不可混淆的保证

Checkpoint 不保证外部操作恰好执行一次；UUID lease_token 不是单调 fencing token；两份 SQLite 没有跨库原子提交。回答时明确工程用幂等、回执、版本检查与恢复作业弥补哪些窗口，仍有哪些限制。

按轮次导航

一、字节跳动 · 一面 / 17 题

一、字节跳动 · 二面 / 22 题

一、字节跳动 · 三面 / 16 题

二、千问 · 一面 / 39 题

二、千问 · 二面 / 26 题

二、千问 · 三面 / 27 题

三、淘天 · 一面 / 34 题

三、淘天 · 二面 / 31 题

四、小红书 · 方向1一面 / 25 题

四、小红书 · 方向1二面 / 20 题

四、小红书 · 方向1三面 / 24 题

四、小红书 · 方向2一面 / 19 题

四、小红书 · 方向2二面 / 27 题

五、美团 · 一面 / 37 题

六、拼多多 · 一面 / 37 题

七、腾讯 · 一面 / 23 题

八、蚂蚁 · 一面 / 43 题

八、蚂蚁 · 二面 / 38 题

九、百度 · 一面 / 34 题

九、百度 · 二面 / 26 题

九、百度 · 三面 / 29 题

十、深信服 · 一面 / 14 题

十一、同花顺 · 一面 / 34 题

附录 A：编码练习与复杂度；附录 B：备考与证据索引。

一、字节跳动 · 一面

1. 为什么选择 LangGraph？与其他 Agent 框架相比，看中了哪些能力？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

2. ReAct 循环在什么情况下会迟迟不结束？

答：ReAct 若反复选相同工具、拿不到成功信号、失败无限重试，或压缩丢失目标就会不收敛。设计上要同时限制步骤、模型调用、总时间和输出大小，并检测无进展。工程当前是固定图，默认执行期限 120 秒、模型超时 45 秒、安全分类 8 秒，且有累计调用/字节额度；可选信息最多追问两轮，必填缺失不能为了结束而保存空记录。

依据 / 对照：backend/app/common/config.py；backend/app/common/execution_budget.py；backend/app/workflows/runtime.py

3. 追问：如何限制循环次数、上下文规模和工具调用范围？

答：ReAct 若反复选相同工具、拿不到成功信号、失败无限重试，或压缩丢失目标就会不收敛。设计上要同时限制步骤、模型调用、总时间和输出大小，并检测无进展。工程当前是固定图，默认执行期限 120 秒、模型超时 45 秒、安全分类 8 秒，且有累计调用/字节额度；可选信息最多追问两轮，必填缺失不能为了结束而保存空记录。

依据 / 对照：backend/app/common/config.py；backend/app/common/execution_budget.py；backend/app/workflows/runtime.py

4. Tool Calling 的协议怎么设计？模型看到的工具描述与实际执行实现如何对应？

答：本项目没有开放式 Tool Calling 注册中心。模型输出固定 Pydantic 结构，程序只接受批准的动作和视图，再由本地函数执行。若扩展工具，注册表应绑定 name/version、输入输出 schema、执行器、权限、超时、幂等策略和副作用等级；模型看到的描述与执行器必须来自同一注册条目，调用前再校验参数与权限，不能只靠描述约束。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

5. Agent 执行到一半进程崩溃，恢复后怎样找到正确的执行位置？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

6. 追问：如果外部工具已经执行过了，恢复时怎么避免重复调用、重复产生副作用？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

7. 面对理解需求、检索资料、生成报告等多步骤任务，会选择单 Agent 加工具，还是多 Agent？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

8. 追问：Manager-Worker 这样的架构适合什么情况？与单 Agent 的取舍是什么？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

9. Agent 运行时记录了哪些 Metrics、Trace 和 Log？

答：运行出口只存固定事件、耗时、结果及关联 ID，不记健康原文、prompt、模型响应、SQL 参数或密钥。日志有轮转/保留，图执行关闭第三方内容 tracing；内部 metrics 默认关闭。质量账本随业务事务去重和删除；运行计数重启会重置，异步日志硬退出可能丢失。当前未接外部遥测平台或按角色的日志查看授权。

依据 / 对照：docs/可观测性与健康数据保护开发方案.md；backend/app/observability/

10. RAG 使用什么向量存储？

答：项目对健康记录使用 SQLite 固定视图查询，没有文档上传知识库、Embedding、向量数据库或 Reranker。扩展方案是解析保留层级/来源，切分建索引，权限过滤后召回、融合、重排，按预算组装证据再回答。小规模明确日期/类型/关键词查询先用结构化检索，不能为了面试把事件检索说成已上线混合 RAG。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

11. 检索返回了很多候选片段，但上下文空间有限，如何排序、压缩和组装？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

12. Agent 如何部署？逻辑上的 Agent 与进程、Worker 是什么关系？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

13. 追问：每个 Agent 都需要一个独立运行实例吗？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

14. 后端平台与 AI 平台如何协作，哪些能力可以复用？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

15. 如何统一接入不同的模型 Provider？

答：工程通过配置 base_url、model_name 和密钥接入兼容 Chat Completions 的接口，固定任务 schema 再用 Pydantic 校验响应。模型可替换不代表所有供应商都已兼容：流式、工具语法、限流和 usage 需单独适配与契约测试。当前没有多 Provider 自动路由、流式协议归一或已启用的 Token 成本统计，不声称已具备完整模型网关。

依据 / 对照：backend/app/modeling/transport.py；backend/app/modeling/transport.py；backend/app/common/config.py

16. 追问：不同 Provider 的流式协议、限流、降级和成本统计怎么处理？

答：工程通过配置 base_url、model_name 和密钥接入兼容 Chat Completions 的接口，固定任务 schema 再用 Pydantic 校验响应。模型可替换不代表所有供应商都已兼容：流式、工具语法、限流和 usage 需单独适配与契约测试。当前没有多 Provider 自动路由、流式协议归一或已启用的 Token 成本统计，不声称已具备完整模型网关。

依据 / 对照：backend/app/modeling/transport.py；backend/app/modeling/transport.py；backend/app/common/config.py

17. 【算法题】 Two Sum 类问题，如何通过一次遍历找到目标组合？题面信息不完整，不还原额外约束。

答：经典版本用哈希表保存已访问值到下标；遍历 x 时先查 target-x，再写 x，避免同一元素重复使用。期望时间 O(n)、空间 O(n)。原题额外约束不完整，附录示例仅演示经典“两下标”题。

一、字节跳动 · 二面

1. 为什么考虑换工作？目前的工作地点和职级情况如何？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

2. AI 应用平台最初为什么立项，面向什么用户？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

3. 追问：现在有哪些真实使用场景，用户实际怎么使用？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

4. 用户在平台上创建的应用如何发布、嵌入或集成到已有系统？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

5. 如果从平台研发转向客户现场交付，可以沉淀哪些方法和可复用资产？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

6. 通用运行内核、业务扩展与服务入口各负责什么？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

7. 业务扩展由谁开发？客户最终买到的是产品、服务还是一套定制应用？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

8. 云端部署与私有化部署分别怎么做？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

9. 追问：客户的数据不能出内网时，模型部署和调用链路如何处理？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

10. 工具在平台中怎样抽象？能否讲清一次工具调用的链路？

答：本项目没有开放式 Tool Calling 注册中心。模型输出固定 Pydantic 结构，程序只接受批准的动作和视图，再由本地函数执行。若扩展工具，注册表应绑定 name/version、输入输出 schema、执行器、权限、超时、幂等策略和副作用等级；模型看到的描述与执行器必须来自同一注册条目，调用前再校验参数与权限，不能只靠描述约束。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

11. Workflow 的工具节点和 ReAct 的 Tool Calling 能否复用同一套执行能力？

答：可以共用同一个工具注册表、参数校验器、授权和执行器；差别在控制流：Workflow 由固定边选择节点，ReAct 由模型建议下一调用。本工程尚无通用工具层，但营养准备函数由“追问期间保存独立记录”和“最终保存”共用，展示了复用能力而不重复业务逻辑的做法。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

12. 运行时对 LangGraph 的依赖有多深？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

13. 追问：Python 性能是否会成为瓶颈？

答：async 协程适合可等待 I/O，不会自动把同步阻塞函数变成非阻塞。同步端点可在线程池执行，CPU 密集任务考虑进程或独立任务服务；普通带 GIL 的 CPython 线程不能让纯 Python CPU 代码多核并行，需区分 free-threaded 构建。当前 Engine 还有锁和 SQLite 单所有者约束，单凭 FastAPI/async 不能推算商业容量。

依据 / 对照：backend/app/main.py；backend/app/workflows/concurrency.py；Python 官方 asyncio 文档

14. 用户身份和角色如何传递到运行时？工具和 Skill 的可见范围、调用权限怎样控制？

答：模型不能提交任意 SQL、字段或记录 ID；本地先绑定目标、批准固定视图，再用 ModelBoundary 发一次性授权，消费前累计计数。processing_plan 仅接收当前输入和操作状态，后续任务按需读取最少数据；local_only 禁止全部外发且随工作流恢复保留。当前没有多用户身份体系，不能把数据最小化等同于多租户鉴权。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/modeling/payloads.py；docs/模型数据访问边界.md

15. 日志、Trace 和指标存在哪里？不同人员如何获得查看和排障权限？

答：运行出口只存固定事件、耗时、结果及关联 ID，不记健康原文、prompt、模型响应、SQL 参数或密钥。日志有轮转/保留，图执行关闭第三方内容 tracing；内部 metrics 默认关闭。质量账本随业务事务去重和删除；运行计数重启会重置，异步日志硬退出可能丢失。当前未接外部遥测平台或按角色的日志查看授权。

依据 / 对照：docs/可观测性与健康数据保护开发方案.md；backend/app/observability/

16. 排障效率的提升具体怎么测量？

答：可引用仓库已有报告而非本次实测：2026-10-27 发布阶段合成本地体重工作流，关闭/开启观测的 P95 为 83.77/87.83 ms，约增加 4.85%；按每模式 200 个样本、交替轮次的报告口径，模型网络与调度关闭。这只能说明该环境的观测开销，不能作线上端到端延迟、TTFT、QPS 或容量承诺；也没有排障效率提升的对照数据。

依据 / 对照：docs/evidence/observability-validation-summary.json

17. 追问：比较基线和统计范围是什么，如何证明是这次改动带来的提升？

答：可引用仓库已有报告而非本次实测：2026-10-27 发布阶段合成本地体重工作流，关闭/开启观测的 P95 为 83.77/87.83 ms，约增加 4.85%；按每模式 200 个样本、交替轮次的报告口径，模型网络与调度关闭。这只能说明该环境的观测开销，不能作线上端到端延迟、TTFT、QPS 或容量承诺；也没有排障效率提升的对照数据。

依据 / 对照：docs/evidence/observability-validation-summary.json

18. 是否考虑过建设统一控制面？当前为什么做或不做？

答：当前设置、计划与数据管理页面承担部分配置职责，API/Engine 执行业务，尚无独立统一控制面。若产品化，控制面管理能力版本、发布、租户权限、配额和灰度；运行面只执行已批准版本，维护任务状态与回执。配置变更须版本化并定义对运行中任务的影响，不能用覆盖配置代替任务迁移。

依据 / 对照：backend/app/main.py；miniprogram/src/pages/；后半为设计

19. 你在项目中具体负责哪些部分？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

20. 行业 Agent 实际解决了什么问题，交付物是什么？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

21. 托管式 Agent Runtime 是否可行，客户会因为什么选择它？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

22. 【算法题】 将两个有序数组原地合并到具有预留空间的第一个数组中。

答：从两数组尾部用双指针取较大值，写入第一个数组预留空间的末尾，避免覆盖尚未读取的元素；第二数组剩余项继续拷贝。时间 O(m+n)、额外空间 O(1)，附录有实现及空数组测试。

一、字节跳动 · 三面

1. AI 应用平台的整体架构怎么设计，采用了哪些技术？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

2. 为什么选择 LangGraph？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

3. DSL 如何定义，又如何转换为执行图？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

4. LangGraph 使用过程中遇到过什么问题？

答：工程处理过的语义难点包括 interrupt 恢复会重跑节点、两库提交窗口、取消/删除期间模型结果晚到。对应采用保存前幂等键、恢复回执、GuardedSqliteSaver 与 epoch/generation 校验。可以结合 test_resume_recovery.py 和 test_workflow_concurrency.py 讲解，但这些是仓库可验证行为，不自动构成个人故障经历。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

5. 追问：是否做过性能测试，系统容量如何？

答：可引用仓库已有报告而非本次实测：2026-10-27 发布阶段合成本地体重工作流，关闭/开启观测的 P95 为 83.77/87.83 ms，约增加 4.85%；按每模式 200 个样本、交替轮次的报告口径，模型网络与调度关闭。这只能说明该环境的观测开销，不能作线上端到端延迟、TTFT、QPS 或容量承诺；也没有排障效率提升的对照数据。

依据 / 对照：docs/evidence/observability-validation-summary.json

6. 平台质量怎么衡量，主要看哪些指标？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

7. 平台后续如何演进？业务 Agent 是否直接建立在原平台之上？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

8. LangGraph 之外，团队自行实现了哪些能力？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

9. 运行时怎样划分职责，通用执行层与业务扩展如何分工？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

10. 【工程编码题】 写出 ReAct Loop 伪代码，并说明如何通过 Hook 接入 Skill、Memory 等能力。

答：核心是构造当前上下文→模型给 Final 或工具调用→本地验参/鉴权→工具执行→记录 Observation→继续。预算、超时、无进展检测和完成校验必须在循环外层强制；Skill/Memory 用 before_model、after_tool、before_finish 等 Hook 注入。附录是练习伪代码，当前工程采用固定图。

11. Memory 在什么时候检索、更新和保存？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

12. 工具调用超时怎么处理？

答：错误区分输入/权限/冲突、模型上游、预算和运行故障，HTTP 层输出固定错误码和关联信息。可重试失败要有次数与退避；参数错或权限拒绝不能盲重试。模型解析失败明确报错；营养估算失败保存营养未知；表达选择失败使用本地合法选项。三类降级语义不同，不能把失败偷偷改成演示成功。

依据 / 对照：backend/app/common/errors.py；backend/app/modeling/errors.py；backend/app/modeling/transport.py；backend/tests/test_error_contract.py

13. 页面关闭后，后台任务如何继续执行？

答：周期扫描负责生成持久化任务，runner 领取、续租、重试并提交结果；周报经 outbox 创建幂等计划任务。页面关闭不影响仍在运行的服务生成站内消息，但停止服务期间不会实时提醒。重启可补最近七个有记录日期的日报，过期提醒不会无限补发。独立 Engine 服务与多实例 Worker 仍属后续阶段。

依据 / 对照：backend/app/jobs/planner.py；backend/app/jobs/runner.py；backend/app/jobs/handlers.py

14. 多个 Worker 怎么避免执行冲突？

答：任务领取在事务内用 status=pending 与 attempts 条件更新，只允许成功者执行；每次领取生成随机 UUID lease_token，并增加 attempt。写入及完成时校验 token、generation、状态和租约未过期，拒绝旧执行者。这里 UUID 是相等性凭证，并非单调 fencing token；若对外部存储做 fencing，需要权威存储发放单调版本，且外部接收端比较版本拒绝旧写入。

依据 / 对照：backend/app/infrastructure/repositories/jobs.py；backend/app/jobs/context.py

15. 如何看待 Agent 与 Harness 的后续演进？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

16. 目前的职级是什么？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

二、千问 · 一面

1. 介绍一下当前工作和比较擅长的方向。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 对工作地点是否有要求，能否接受异地机会？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

3. AI 应用平台与业务 Agent 项目是什么关系？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

4. 你在其中具体承担哪些工作？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

5. 为什么需要单独建设 Harness，它要解决什么问题？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

6. 企业 Agent 除了 ReAct、Tool Calling 和上下文管理，还需要补哪些能力？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

7. 通用运行内核、业务扩展与服务入口如何划分？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

8. 业务扩展模块具体包含什么？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

9. 这些模块如何注册、管理版本和升级？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

10. 运行内核如何与具体业务实现解耦？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

11. Memory 策略应该由业务决定，还是固定在运行内核中？

答：策略应由业务决定，内核提供读取/写入时机、预算、授权与来源契约。本项目健康偏好、过敏、期限和冲突处理放在 health/facts.py 与 fact_resolution.py；工作流负责何时提取、何时校验和保存。当前采用模块装配，还没有可替换 Memory 插件协议。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

12. 能力声明文件需要描述哪些信息？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

13. Tool、Skill、Artifact 和 Hook 如何组织？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

14. Workflow 与 ReAct 如何配合？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

15. 业务扩展到什么程度应当修改内核，边界在哪里？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

16. 目前有多少类业务扩展，是否经过不同场景验证？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

17. 多人同时开发不同业务能力时，怎样协作和集成？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

18. 不同业务独立部署，还是共享运行时？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

19. 追问：共享时怎么做隔离？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

20. Skill 如何路由和选择？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

21. 用户怎样发现和使用已有 Skill？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

22. Skill 是代码实现还是配置定义？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

23. 是否提供沙箱，主要解决什么问题？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

24. 沙箱带来的延迟与安全收益怎么权衡？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

25. Serverless 与临时创建 Kubernetes Pod 的方式有什么差别？

答：项目只有常驻服务与 Compose 加固配置，没有临时 Pod 或 Serverless。设计上 Serverless 倾向按请求/事件调度且受平台生命周期限制；临时 Pod 可显式配置镜像、网络和资源，但有调度/拉镜像成本。状态都应放外部持久库；不可信代码隔离强度取决于实际运行时，不能仅按产品名称判断。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

26. 目前的用户量和并发规模如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

27. 实时交互与后台长任务如何安排 Worker？

答：周期扫描负责生成持久化任务，runner 领取、续租、重试并提交结果；周报经 outbox 创建幂等计划任务。页面关闭不影响仍在运行的服务生成站内消息，但停止服务期间不会实时提醒。重启可补最近七个有记录日期的日报，过期提醒不会无限补发。独立 Engine 服务与多实例 Worker 仍属后续阶段。

依据 / 对照：backend/app/jobs/planner.py；backend/app/jobs/runner.py；backend/app/jobs/handlers.py

28. 你在平台项目中负责哪些模块？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

29. 平台的实际使用规模怎样，是否已有线上用户？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

30. 应用如何发布和集成到外部系统？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

31. 底层执行引擎是什么？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

32. 画布 DSL 如何转换为执行图？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

33. 是否比较过 AgentScope、pi-agent 等框架，各自有什么特点？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

34. ToB 与 ToC Agent 的主要区别是什么？

答：ToB 更常强调组织权限、审计、系统集成和可验收流程；ToC 更强调个人偏好、交互延迟、长期记忆和成本，但二者都需要隐私保护。本项目属于单用户 ToC 原型，所以结构化个人事实、追问恢复和可删除性优先；企业租户、组织角色与连接器尚未实现。

依据 / 对照：README.md；miniprogram/README.md

35. 面向 ToC 时，Memory 应该怎样设计？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

36. 工具越来越多，会带来哪些上下文和选择噪声，如何处理？

答：工程优先不发送无关内容：首轮不带最近聊天和健康历史，后续只发经批准的固定视图。咨询饮食汇总最多近 7 天，体重/分量上下文最多近 28 天；明确更短范围就按更短范围。关键日期、数值、来源、未知状态和 local_only 必须保留。当前有字节和调用预算，未实现按 tokenizer 的动态 Token 分配或模型摘要压缩。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

37. Skill 规则与 Memory 冲突时怎么办？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

38. Python 和 Java 分别使用到什么程度？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

39. 如果团队使用另一种语言，准备怎样适应？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

二、千问 · 二面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 最近在读哪些 Agent 框架？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

3. 是否看过 Coding Agent 的实现，重点研究了哪些部分？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

4. 怎样判断一个 Harness 好不好？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

5. 自己的运行时实现了什么，能力边界在哪里？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

6. 通用运行内核、业务扩展和服务入口怎样分层？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

7. Tool、Skill 和 Memory 的接口分别如何设计？

答：Tool 接口应描述 name/version、schema、权限、执行结果与幂等；Skill 描述目标、适用范围、资源和流程；Memory 提供按授权和时间范围读写、版本、来源、撤回与冲突结果。本工程目前是固定模型任务 + Python 图 + SQLite 事实，并未统一暴露这三种可插拔接口。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

8. 运行时的输入和输出是什么？

答：输入不只是自然语言，还包括 request_id、conversation_id、processing_mode，以及恢复时的 thread_id、interrupt_id、补充文本或 approved。状态保存草稿、待补字段、已保存索引、安全判断和计划等。输出包含回复、记录卡片、证据引用、流程状态/待输入信息和实际模型发送披露；这些是有契约的业务结果，不是任意文本。

依据 / 对照：backend/app/workflows/state.py；backend/app/workflows/runtime.py

9. 多个业务并行扩展时，怎样避免互相影响？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

10. 如何评估自研系统与开源方案的差异？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

11. 追问：是否真正用相同任务对比过通用 Agent？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

12. 测试用例、Ground Truth 和 A/B 对比具体怎么做？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

13. 当前使用哪些模型？

答：模型由 MODEL_BASE_URL、MODEL_NAME 和密钥显式配置，支持兼容 Chat Completions 的服务；源码与文档有 Qwen 相关适配。实际使用哪一个需以运行时配置和评测报告为准，不读取个人 .env 或凭据，也不把某次实验模型说成所有环境固定配置。全部未配置进入明确演示模式，部分配置缺失则启动失败。

依据 / 对照：backend/app/modeling/transport.py；backend/app/modeling/transport.py；backend/app/common/config.py

14. 是否做过模型后训练？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

15. 框架给开发者提供什么接口，开发者需要写哪些代码？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

16. 与 AgentScope 相比，有什么不同？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

17. 是否了解托管式 Agent 服务？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

18. 目前有多少真实用户，实际使用情况如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

19. 开发中遇到的最大挑战是什么？

答：代码与测试体现的一个问题是：“帮我记录昨天吃得有点咸”缺少具体食物，不能写空餐食，也不能在后续补充时丢掉原日期。方案是先提取允许不完整的草稿，本地判断必填字段，interrupt 持久化待补项，恢复仅 patch 当前字段，并用日期/证据/重放测试验收。这是可从仓库复现的技术案例，不据此虚构个人线上故障或收益。

依据 / 对照：backend/tests/test_draft_clarification.py；backend/tests/test_contextual_replies.py；backend/app/workflows/chat_flow.py

20. 较重的 Workflow 如何与 ReAct 主循环结合？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

21. Sub-agent 如何定义和启动？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

22. 追问：子 Agent 是提前声明的，还是运行过程中动态创建的？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

23. 为什么考虑离开当前公司？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

24. 为什么选择这个方向和这家公司？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

25. 如何看待当前公司的优势与不足？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

26. 后续职业规划是什么？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

二、千问 · 三面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 入职当前公司时是什么级别，现在是什么级别？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

3. 是否有晋升经历？如有，最近一次是什么时候？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

4. 追问：如果有晋升经历，主要凭借哪些工作和贡献？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

5. 如果有晋升和转向 AI 研发的经历，两者的时间先后关系是什么？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

6. 介绍一下当前业务 Agent 项目。

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

7. FDE 在实际工作中意味着什么？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

8. 项目团队有哪些角色，怎样分工？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

9. 是否所有研发人员都参与运行内核开发？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

10. 业务 Agent 是否建立在原有应用平台之上？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

11. 你具体负责哪部分？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

12. 架构方案由谁决定，你参与了哪些设计决策？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

13. 为什么采用运行内核、业务扩展和服务入口的分层方式？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

14. 运行内核内部包含哪些模块？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

15. Workflow 和 ReAct 如何结合？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

16. 除此之外，还有哪些比较有特点的设计？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

17. 为什么把 Memory 实现放在运行内核之外？

答：因为健康领域的“应记什么、冲突怎么处理、何时失效”属于业务语义。checkpoint 只保存当前工作状态；长期事实放业务 SQLite 并由领域层负责修订和撤回。分开后能独立治理生命周期，但当前仍是项目内模块，不应说已实现完全独立的 Memory 服务。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

18. 记忆接口如何定义？

答：当前 Repository/领域函数提供事件、事实、派生结果与修订查询，工作流保存 fact_id 和证据引用。若抽象接口，read 应接主体、目的、日期/范围与预算并返回来源版本；write 应含陈述类型、有效期、证据、幂等键和预期版本，并返回生效/冲突/待确认状态。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

19. 追问：具体实现由谁提供，又怎样注入运行时？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

20. 上下文工程做了哪些工作？

答：工程优先不发送无关内容：首轮不带最近聊天和健康历史，后续只发经批准的固定视图。咨询饮食汇总最多近 7 天，体重/分量上下文最多近 28 天；明确更短范围就按更短范围。关键日期、数值、来源、未知状态和 local_only 必须保留。当前有字节和调用预算，未实现按 tokenizer 的动态 Token 分配或模型摘要压缩。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

21. Token 预算如何分配，什么时候触发压缩？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

22. 工具结果和历史消息分别怎么压缩？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

23. 这些设计中，哪些已经做过实际验证？

答：仓库 2026-10-26 报告共 16 条合成语义样本：10 条风险均被拦截、6 条普通均放行，但其中 2 条风险首选路由与标注不同。可说该小样本的拦截结果，不可说分类完全正确或临床有效。回归另用模拟模型检查超时、非法证据、local_only、恢复与计划绕过；本次文档整理未重跑供应商评测。

依据 / 对照：docs/evidence/safety-evaluation-2026-10-26.json；backend/scripts/evaluate_safety.py

24. 平时怎样追踪 Agent 领域的变化？

答：个人阅读习惯需据实补充。可采用固定方法：跟一条请求读入口、状态、模型授权、执行、恢复和测试；对新机制用同一合成任务做对照，不只追框架名。当前项目最值得持续核验的是持久化语义、模型接口兼容、安全分类与隐私删除，而非未经试验就更换架构。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

25. 最近观察到哪些值得关注的架构趋势？

答：可从本项目推演的方向是：更多模型建议由本地执行边界约束、持久化长任务、结构化记忆及按任务评测。这是基于项目需求的判断，不代表行业最新趋势结论。讨论新架构时应明确它改善了哪个失败模式，并用同条件实验验证。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

26. 下一阶段的职业发展考虑是什么？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

27. 工作地点上有哪些考虑？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

三、淘天 · 一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 单 Agent、多 Agent 和 Workflow 分别适合什么场景？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

3. 它们的核心区别是什么？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

4. 任务复杂，就一定应该用多 Agent 吗？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

5. 追问：什么条件下才值得引入多 Agent？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

6. 为什么将 Workflow 封装为 Skill？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

7. 一次用户请求从进入到结束，生命周期是什么样的？

答：小程序提交 request_id 和处理模式；服务端检查回执与指纹，定位或创建 thread；安全入口先判定允许能力；模型解析草稿，本地校验缺失字段与权限；营养准备后保存完整独立记录，缺项则 interrupt 追问；最终本地渲染结果并保存回执。恢复时校验 thread、interrupt_id 和请求指纹，再继续原图。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py

8. 运行内核、业务扩展和服务入口分别负责什么？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

9. 一次完整任务与其中一轮模型推理、工具调用是什么关系？

答：一次任务可跨多轮模型推理、多个本地动作和多次人工补充；它的终态由业务验收决定。单轮推理只给结构化建议，工具调用只完成一个动作。工程用 thread_id 关联任务，request_id 去重某次请求，interrupt_id 绑定等待点，不能把任意一次模型 HTTP 返回当作任务完成。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

10. Skill 如何被选中，并进入上下文？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

11. 工具调用什么时候并行，什么时候串行？

答：没有数据依赖、权限独立且副作用不冲突的只读操作可以并行；后一步依赖前一步结果、共享可变事实或确认边界时应串行。本项目聊天图主要顺序执行，以确定性保存为主；并行化前要测实际收益，并在汇聚时校验来源版本，不能仅用 Promise/gather 包一层就宣称安全。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

12. 为什么选择 LangGraph？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

13. Agent 效果从哪些维度评估？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

14. 测试用例和参考执行轨迹怎么构建？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

15. LLM-as-a-Judge 具体评价哪些内容？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

16. 怎样把评测接入发布和回归流程？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

17. 如果上线后效果下降，怎么定位和回滚？

答：先固定失败样本，按解析、授权、数据视图、模型输出与本地渲染逐层定位，并比较模型/提示/策略/数据版本。发布前用合成回归和安全门禁，灰度时分组监控任务完成与越权；回滚需保留兼容状态和数据库迁移方案。工程已有测试与版本化派生结果，没有完整线上 A/B 和自动回滚平台。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

18. 如何防范 Prompt Injection？

答：把用户输入、检索文本和模型结果视为数据；执行层只接受固定 schema 和动作白名单，重新检查授权、版本与业务规则。工程在解析、估算、保存、回复和计划激活前做安全检查，输出日期、数值和保存状态由本地绑定。即使模型说“跳过审批”也无权激活计划；Prompt 禁令只是补充，不能替代执行检查。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/workflows/safety.py；backend/app/policy/output.py

19. 沙箱承担哪些安全职责？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

20. Agent 需要访问敏感数据时，怎样控制权限？

答：模型不能提交任意 SQL、字段或记录 ID；本地先绑定目标、批准固定视图，再用 ModelBoundary 发一次性授权，消费前累计计数。processing_plan 仅接收当前输入和操作状态，后续任务按需读取最少数据；local_only 禁止全部外发且随工作流恢复保留。当前没有多用户身份体系，不能把数据最小化等同于多租户鉴权。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/modeling/payloads.py；docs/模型数据访问边界.md

21. 上下文接近模型窗口上限时怎么办？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

22. 平时如何保持上下文干净，避免无关信息不断积累？

答：工程优先不发送无关内容：首轮不带最近聊天和健康历史，后续只发经批准的固定视图。咨询饮食汇总最多近 7 天，体重/分量上下文最多近 28 天；明确更短范围就按更短范围。关键日期、数值、来源、未知状态和 local_only 必须保留。当前有字节和调用预算，未实现按 tokenizer 的动态 Token 分配或模型摘要压缩。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

23. 工具返回过长，或连续调用工具产生大量记录时，怎么处理？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

24. 追问：压缩时哪些信息必须保留？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

25. 业务场景中的长期记忆如何组织？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

26. 新旧记忆冲突时怎么处理？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

27. 【工程编码题】 实现一个简化的 Plan-and-Execute Agent：查询两个城市的天气，并计算温度差。

答：先校验步骤类型、依赖和引用，再查询两个城市的天气，温差由本地代码计算，Summarizer 只读已确认结果。附录提供可运行模拟天气示例，明确 tool 与 compute 类型；不调用真实天气 API，也不声称本项目已有通用计划执行器。

28. Planner、Executor 和 Summarizer 各负责什么？

答：Planner 只生成有类型的步骤与依赖，不执行；Executor 校验工具、参数、权限和前序引用后执行并保存结果；Summarizer 只读已确认结果来表达。本项目的对应思想是模型给 processing_plan、本地审查执行、输出由本地绑定。天气题参考编码附录，属于独立练习，不是现有业务。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

29. 步骤怎样表示，后续步骤如何引用前序结果？

答：步骤可表示为 {id, kind, capability, args, depends_on}；参数引用使用受限 {ref: step_id, field: temperature_c}，由执行器解析，不能 eval 字符串。执行前检查依赖存在、无环、引用字段和类型；结果存按 step_id 索引的结构化表。当前工程没有通用 DAG 计划解释器，附录提供天气练习。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

30. 怎样区分调用工具的步骤和调用模型的步骤？

答：使用显式 kind=tool/model/compute，并为每种类型绑定不同执行器和权限。天气查询是 tool，本地温差计算是 compute，最终自然语言表达可选 model。不要让模型通过参数偷偷指定 shell、SQL 或任意 URL；本项目实际使用固定模型任务和本地函数，没有动态工具注册执行器。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

31. 工具注册表如何设计？

答：本项目没有开放式 Tool Calling 注册中心。模型输出固定 Pydantic 结构，程序只接受批准的动作和视图，再由本地函数执行。若扩展工具，注册表应绑定 name/version、输入输出 schema、执行器、权限、超时、幂等策略和副作用等级；模型看到的描述与执行器必须来自同一注册条目，调用前再校验参数与权限，不能只靠描述约束。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

32. 目前其他求职流程进展如何？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

33. 对工作地点有什么考虑？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

34. 为什么考虑换工作？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

三、淘天 · 二面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. AI 应用平台最初要解决什么问题，为什么建设这个平台？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

3. 与 Dify、Coze、Palantir 等方案相比，借鉴了什么，又有什么不同？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

4. 是否阅读过相关开源项目的源码？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

5. Workflow 节点非常多时，怎样处理复杂度？

答：当前固定图按 safety、parsing、nutrition、persist、reply 等模块分工。若画布节点很多，按稳定输入输出拆子流程，限制单节点职责，版本化 schema，提供编译期引用/连线检查与运行追踪；不能靠把图缩成一个巨大代码节点掩盖复杂度。本项目没有画布产品。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

6. 是否支持通过自然语言生成 Workflow？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

7. 为什么不让模型直接生成最终 DSL？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

8. 生成后怎样校验节点、参数和连线？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

9. 能否自动执行测试，并根据错误修复生成结果？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

10. 工具数量很多时，如何组织和选择？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

11. 如果是面向 ToC 的 Agent，工具信息如何进入上下文？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

12. 如何通过作用域和渐进式披露限制当前可见能力？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

13. 工具描述和历史消息过长时如何压缩？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

14. 完整的 RAG 链路怎么设计？

答：项目对健康记录使用 SQLite 固定视图查询，没有文档上传知识库、Embedding、向量数据库或 Reranker。扩展方案是解析保留层级/来源，切分建索引，权限过滤后召回、融合、重排，按预算组装证据再回答。小规模明确日期/类型/关键词查询先用结构化检索，不能为了面试把事件检索说成已上线混合 RAG。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

15. 不同文件类型分别如何处理？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

16. PPL 切分是怎么做的？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

17. BM25、向量检索、RRF、MMR 和 Rerank 如何组合？

答：BM25 补关键词精确匹配，向量补语义召回；分别取候选，RRF 按名次融合而非直接相加不同量纲分数，再去重、用 Reranker 排相关性。MMR 在相关性与结果多样性间权衡，适合候选重复时选用；若降低证据完整性或延迟不值就不启用。本项目未使用这些组件，具体收益必须由固定数据集消融验证。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

18. 如何评估检索和回答效果？

答：Recall@K 看已标注相关证据召回覆盖，MRR 看第一个相关结果的位置，NDCG 看带相关等级的排序质量；多证据问答还应检查证据集合是否齐全。回答侧单独测正确性、依据一致性、拒答和引用定位，联合记录延迟/成本。项目尚无 RAG 指标实测，不能引用安全分类测试替代检索评测。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

19. 如果生成答案和评价答案都依赖同类模型，怎样看待评测偏差？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

20. 模型生成的评测集可信吗，怎样验证？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

21. 在已有平台之外，为什么还需要独立的业务 Harness？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

22. 运行内核、业务扩展与服务入口怎样分工？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

23. 这套运行时能否迁移到其他行业？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

24. 不同行业的业务指标不同，通用能力和业务评价怎么分开？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

25. 页面关闭或服务重启后，任务如何恢复？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

26. 如何认领任务，避免重复执行？

答：任务领取在事务内用 status=pending 与 attempts 条件更新，只允许成功者执行；每次领取生成随机 UUID lease_token，并增加 attempt。写入及完成时校验 token、generation、状态和租约未过期，拒绝旧执行者。这里 UUID 是相等性凭证，并非单调 fencing token；若对外部存储做 fencing，需要权威存储发放单调版本，且外部接收端比较版本拒绝旧写入。

依据 / 对照：backend/app/infrastructure/repositories/jobs.py；backend/app/jobs/context.py

27. 追问：如果工具会产生外部副作用，怎样处理重复调用？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

28. 从业务模板演进到通用运行时，需要做哪些调整？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

29. 这套设计迁移到 ToC 场景，会遇到什么问题？

答：本项目本来就是个人 ToC 原型；若从企业共享平台迁移到个人场景，主要新增长期偏好治理、轻量交互、成本和隐私删除要求，同时仍须隔离用户。不能把本项目反向描述成已完成企业平台到 ToC 的迁移经验。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

30. 为什么考虑离开当前公司？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

31. 其他求职流程进展如何？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

四、小红书 · 方向1一面

1. 平台的定位是什么，核心能力有哪些？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

2. 客户实际怎样使用这个平台？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

3. 同时支持 Workflow 和自主 Agent 吗？应用如何发布？

答：当前支持固定 LangGraph 工作流和模型生成的受限处理计划，没有开放自治 Agent 或用户画布。小程序独立 build:weapp 后由微信开发者工具构建上传，API 单独部署。图和业务规则随服务版本发布，不支持在线发布自定义 Agent。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

4. 平台与现有产品相比，有什么差异化？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

5. 为什么用 LangGraph，而不是沿用原有后端语言自行实现？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

6. 多 Agent 采用什么协作模式？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

7. Planner 如何选择和路由到具体 Agent？

答：当前没有多 Agent Planner。扩展时先按权限/业务域过滤 Worker，再基于声明的输入输出和任务类型做规则或模型路由；歧义时澄清或单一协调者裁决。选错后不能让 Worker 越权补救，需返回结构化 unsupported 并在总预算内重新规划。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

8. 多个 Agent 能力相似时，会不会选错？

答：能力相似会增加误路由。可用互斥职责、明确正反例、置信不足时澄清、可验证的输出契约降低错误；评测既看路由混淆矩阵，也看最终任务是否完成和额外调用成本。本项目未实现这套多 Agent 路由。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

9. 追问：路由是否正确，怎样评估？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

10. Agent 之间怎样传递上下文？

答：扩展设计应持久化 parent_task_id、child_task_id、目标、权限、输入快照、依赖、状态、结果引用及 continuation。交接不传完整对话；只传已确认事实、剩余任务、失败与预算。恢复先读取依赖图和子任务结果，复用已完成结果，再唤醒父任务；对子任务创建和父任务汇聚都用幂等键。本项目 outbox 可示范可靠创建后续任务，但不等同于通用多 Agent。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

11. 如果并行执行并写入共享状态，怎么处理冲突？

答：编译图可以共享，运行状态由 thread_id 隔离。工程用 request/workflow/conversation 键控锁和 Engine 短临界区协调本地并发；模型 I/O 时释放总锁，返回后核对来源版本、epoch 和 workflow generation，避免旧结果写回。同数据目录还持有 OS 独占锁；这不是多个 Engine 的分布式并发方案。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/infrastructure/data_owner.py；backend/tests/test_workflow_concurrency.py

12. 检索效果用什么指标衡量？

答：Recall@K 看已标注相关证据召回覆盖，MRR 看第一个相关结果的位置，NDCG 看带相关等级的排序质量；多证据问答还应检查证据集合是否齐全。回答侧单独测正确性、依据一致性、拒答和引用定位，联合记录延迟/成本。项目尚无 RAG 指标实测，不能引用安全分类测试替代检索评测。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

13. 评测数据集怎样构建，预期召回结果由谁确定？

答：应由熟悉语料的人员标注 query 对应证据及可接受答案，记录相关等级和必须联合召回的证据集，另一人复核争议项；再分训练/调参/留出集。模型可辅助造题，不能自动成为唯一真值。本项目只有业务与安全合成用例，没有标注好的文档 RAG 集。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

14. 为什么从单一检索演进到混合检索，再加入重排？

答：BM25 补关键词精确匹配，向量补语义召回；分别取候选，RRF 按名次融合而非直接相加不同量纲分数，再去重、用 Reranker 排相关性。MMR 在相关性与结果多样性间权衡，适合候选重复时选用；若降低证据完整性或延迟不值就不启用。本项目未使用这些组件，具体收益必须由固定数据集消融验证。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

15. Query Rewrite 具体做了什么？

答：可用来解析指代、补充必要业务术语和分解多跳查询，但必须保留用户的否定、时间范围与权限。原 query 与改写 query 并行召回可以降低漂移风险；评估同一查询集改写前后 Recall、答案依据、延迟及失败切片。本工程先用本地绑定“这顿”的记录引用，未实现通用查询改写或收益对照。

依据 / 对照：docs/模型数据访问边界.md；后半为设计

16. 追问：提升怎么测出来的，有没有失败案例？

答：可用来解析指代、补充必要业务术语和分解多跳查询，但必须保留用户的否定、时间范围与权限。原 query 与改写 query 并行召回可以降低漂移风险；评估同一查询集改写前后 Recall、答案依据、延迟及失败切片。本工程先用本地绑定“这顿”的记录引用，未实现通用查询改写或收益对照。

依据 / 对照：docs/模型数据访问边界.md；后半为设计

17. 多模态文档如何解析和切分，对检索有什么影响？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

18. PPL 切分的原理是什么？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

19. 用什么模型计算，成本如何？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

20. 切分阈值是固定的还是动态的？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

21. 长对话中出现偏离当前任务的消息，怎样处理边界？

答：先判断是否修改当前任务或开启新任务；可选补充允许“不知道/不记了”结束，必填缺失不强行保存。当前草稿和新工作流隔离，不让一次偏咸反馈自动串到后续餐食；新的输入或恢复回答也要重新通过安全入口。不要把偏题文本直接当作长期偏好。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

22. 短期上下文与长期记忆如何区分？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

23. 长期记忆怎样更新？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

24. Memory 与固定 Workflow 的关系是什么？

答：Workflow 决定何时读取和写入，Memory 提供有版本、有效期与来源的事实；记忆是流程输入，不能修改固定边或绕过确认。工程建议和计划读取按日期解析的事实视图，安全限制优先于普通偏好，派生规律也不会直接改写已确认目标。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

25. 是否支持 Skill？Memory 与 Skill 的规则冲突时怎么办？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

四、小红书 · 方向1二面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 平台的整体架构和核心模块是什么？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

3. 画布 DSL 如何转换成 LangGraph 图？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

4. Workflow 与 ReAct 如何复用底层能力？

答：本项目没有开放式 Tool Calling 注册中心。模型输出固定 Pydantic 结构，程序只接受批准的动作和视图，再由本地函数执行。若扩展工具，注册表应绑定 name/version、输入输出 schema、执行器、权限、超时、幂等策略和副作用等级；模型看到的描述与执行器必须来自同一注册条目，调用前再校验参数与权限，不能只靠描述约束。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

5. 与 OpenClaw 这类 Agent 相比，有什么区别？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

6. 长期记忆和个人工作空间怎样设计？

答：当前长期记忆在业务 SQLite，工作流上下文在 checkpoints.db；没有每个 Agent 的文件工作区。扩展个人空间时把文件、索引、凭据和任务状态绑定用户/租户及授权范围，公共资产只读复用。长期事实读取走版本化视图，不能靠文件路径约定代替权限检查。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

7. 业务数据平台中的数据、逻辑和动作如何接入 Agent？

答：工程使用健康事件、结构化事实和派生结果，没有知识图谱或企业本体平台。扩展可由业务域定义实体、关系、动作和授权语义，运行层只接收版本化查询/动作接口；图查询参数化并限制跳数、返回量和租户范围，结果带实体/来源标识。不能把模型生成的关系直接当作权威事实入库。

依据 / 对照：backend/app/contracts/；backend/app/health/；后半为设计

8. 业务本体由平台统一建设，还是由具体项目提供？

答：工程使用健康事件、结构化事实和派生结果，没有知识图谱或企业本体平台。扩展可由业务域定义实体、关系、动作和授权语义，运行层只接收版本化查询/动作接口；图查询参数化并限制跳数、返回量和租户范围，结果带实体/来源标识。不能把模型生成的关系直接当作权威事实入库。

依据 / 对照：backend/app/contracts/；backend/app/health/；后半为设计

9. 知识图谱怎样查询，查询结果如何用于任务？

答：工程使用健康事件、结构化事实和派生结果，没有知识图谱或企业本体平台。扩展可由业务域定义实体、关系、动作和授权语义，运行层只接收版本化查询/动作接口；图查询参数化并限制跳数、返回量和租户范围，结果带实体/来源标识。不能把模型生成的关系直接当作权威事实入库。

依据 / 对照：backend/app/contracts/；backend/app/health/；后半为设计

10. RAG 是固定执行链路，还是作为 Tool 让模型决定是否调用？

答：两种都可设计：强制先取证的领域流程用固定 RAG 节点；开放探索任务可把检索暴露为受限工具。但二者应复用同一授权与查询服务，并在最终答案验证证据。工程目前用批准的 SQLite 数据视图，不是文档 RAG，也没有让模型自由选择 SQL 或知识库。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

11. 工具返回内容很长时，如何裁剪？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

12. 使用了哪些文档切分策略，各自适合什么情况？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

13. PDF、PPT、Excel 分别如何处理？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

14. 追问：图片和表格怎样解析？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

15. 切分粒度与召回效果之间是什么关系？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

16. 混合检索和重排具体怎么组合？

答：BM25 补关键词精确匹配，向量补语义召回；分别取候选，RRF 按名次融合而非直接相加不同量纲分数，再去重、用 Reranker 排相关性。MMR 在相关性与结果多样性间权衡，适合候选重复时选用；若降低证据完整性或延迟不值就不启用。本项目未使用这些组件，具体收益必须由固定数据集消融验证。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

17. 为什么没有采用 MMR？

答：本项目根本没有向量召回与 MMR，因此不存在“上线后放弃 MMR”的事实。设计取舍是：候选高度重复且多样性有益时考虑 MMR；单证据精确问答或多证据必须齐全时，多样性可能伤害完整性。固定数据集做有无 MMR 对照后再决定。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

18. Workflow 作为 Skill，究竟是什么意思？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

19. Skill、Tool、Workflow 与通用运行内核怎样解耦？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

20. 当前职级和过去的绩效情况如何？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

四、小红书 · 方向1三面

1. 平台主要解决什么问题，核心指标是什么？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

2. 主要部署在公有云还是客户私有环境？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

3. 当前客户、用户以及 Agent / Workflow 的使用规模如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

4. 整体架构怎么设计？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

5. 为什么选 LangGraph，与其他框架比较过吗？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

6. DSL 从加载、编译到执行，完整过程是什么？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

7. 节点类型如何组织，不同能力怎么接入？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

8. 与 Dify、Coze 相比有什么不同？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

9. Workflow 与 ReAct 怎样共用执行能力？

答：本项目没有开放式 Tool Calling 注册中心。模型输出固定 Pydantic 结构，程序只接受批准的动作和视图，再由本地函数执行。若扩展工具，注册表应绑定 name/version、输入输出 schema、执行器、权限、超时、幂等策略和副作用等级；模型看到的描述与执行器必须来自同一注册条目，调用前再校验参数与权限，不能只靠描述约束。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

10. Agent 能否作为一个 Workflow 节点？

答：可以作为设计：把子 Agent 包成受限节点，固定输入输出、预算和权限，调用后返回结构化结果；需要等待就挂起主图，持久化父子关联。当前没有子 Agent，不能称已实现该模式；并行写入与失败传播仍需额外协议。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

11. Worker、Token 和模型资源如何管理？

答：当前配置限制 2 个活跃工作流、后台并发 1、执行期限与模型调用/字节额度；并未启用真实 Token 使用量和成本统计。若扩展共享资源池，要把请求队列、模型并发、每租户额度及预算统一调度，按任务类型设优先级并防止后台任务挤占交互。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

12. 项目最大的挑战是什么，你具体贡献了什么？

答：代码与测试体现的一个问题是：“帮我记录昨天吃得有点咸”缺少具体食物，不能写空餐食，也不能在后续补充时丢掉原日期。方案是先提取允许不完整的草稿，本地判断必填字段，interrupt 持久化待补项，恢复仅 patch 当前字段，并用日期/证据/重放测试验收。这是可从仓库复现的技术案例，不据此虚构个人线上故障或收益。

依据 / 对照：backend/tests/test_draft_clarification.py；backend/tests/test_contextual_replies.py；backend/app/workflows/chat_flow.py

13. 接入 API、MCP、知识库和业务数据分别需要多少开发工作？

答：当前无 MCP、文档知识库集成，无法给实际人日。接口接入需评估鉴权、schema、数据最小化、幂等、错误/超时、删除血缘和测试；只封装 HTTP 请求不是全部工作量。可先做一个最小适配验证，再按共同基础设施与领域增量分别估时。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

14. 为什么引入 Kafka，同步的是什么数据？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

15. 运行时如何跨客户复用？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

16. 业务扩展模块里包含哪些内容？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

17. 上下文如何分层，各层分别放什么？

答：工程优先不发送无关内容：首轮不带最近聊天和健康历史，后续只发经批准的固定视图。咨询饮食汇总最多近 7 天，体重/分量上下文最多近 28 天；明确更短范围就按更短范围。关键日期、数值、来源、未知状态和 local_only 必须保留。当前有字节和调用预算，未实现按 tokenizer 的动态 Token 分配或模型摘要压缩。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

18. 什么时候触发压缩，怎样保留关键内容？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

19. 当前能力处于 PoC 还是生产使用阶段？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

20. PoC 验证和最终交付分别由谁负责？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

21. 长期记忆如何分类、提取和更新？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

22. 多 Agent 如何进行委派、隔离和任务交接？

答：扩展设计应持久化 parent_task_id、child_task_id、目标、权限、输入快照、依赖、状态、结果引用及 continuation。交接不传完整对话；只传已确认事实、剩余任务、失败与预算。恢复先读取依赖图和子任务结果，复用已完成结果，再唤醒父任务；对子任务创建和父任务汇聚都用幂等键。本项目 outbox 可示范可靠创建后续任务，但不等同于通用多 Agent。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

23. 当前职级与入职时相比有什么变化？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

24. 为什么考虑换工作，又为什么选择这个方向？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

四、小红书 · 方向2一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 平台是基于 LangGraph 建设的吗？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

3. 在框架之上，自己补充了哪些能力？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

4. 节点和 DSL 怎样转换为可执行图？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

5. 文件处理、RAG 和评测能力分别做了哪些扩展？

答：文件处理和文档 RAG 未实现；评测方面已有 backend/tests 的合成业务/故障回归及独立安全语料和真实模型评测脚本。可以展示安全、输出、隐私删除与恢复的验证，不能把这些说成文档检索扩展或 Recall 提升。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

6. 为什么不直接使用 Dify？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

7. 平台的定位和能力边界与已有产品有什么不同？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

8. 目前有哪些真实业务在使用？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

9. 平台面向 ToB 还是 ToC？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

10. Agent 生态变化很快，这些变化对平台有什么影响？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

11. 平台更偏用于配置和管理的管理台，还是直接承接在线业务流量的运行台？

答：当前设置、计划与数据管理页面承担部分配置职责，API/Engine 执行业务，尚无独立统一控制面。若产品化，控制面管理能力版本、发布、租户权限、配额和灰度；运行面只执行已批准版本，维护任务状态与回执。配置变更须版本化并定义对运行中任务的影响，不能用覆盖配置代替任务迁移。

依据 / 对照：backend/app/main.py；miniprogram/src/pages/；后半为设计

12. 配置控制面与运行执行面如何划分？

答：当前设置、计划与数据管理页面承担部分配置职责，API/Engine 执行业务，尚无独立统一控制面。若产品化，控制面管理能力版本、发布、租户权限、配额和灰度；运行面只执行已批准版本，维护任务状态与回执。配置变更须版本化并定义对运行中任务的影响，不能用覆盖配置代替任务迁移。

依据 / 对照：backend/app/main.py；miniprogram/src/pages/；后半为设计

13. 开放自治 Agent 与固定 Workflow 应怎样取舍？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

14. 哪些具体业务适合采用 Agent？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

15. 公司内部是否已有相关 SDK 或托管产品，为什么仍需要建设这套能力？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

16. 如果模型跳过权限判断或审批节点，仅靠 Skill 描述能否保证强约束流程？

答：把用户输入、检索文本和模型结果视为数据；执行层只接受固定 schema 和动作白名单，重新检查授权、版本与业务规则。工程在解析、估算、保存、回复和计划激活前做安全检查，输出日期、数值和保存状态由本地绑定。即使模型说“跳过审批”也无权激活计划；Prompt 禁令只是补充，不能替代执行检查。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/workflows/safety.py；backend/app/policy/output.py

17. 必需的工具没有调用、必需的产物没有生成时，怎样判断任务是否完成？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

18. 追问：运行结束时校验不通过，后续怎么处理，是否计为成功？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

19. 对工作地点是否有要求？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

四、小红书 · 方向2二面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 挑一个最有代表性的项目介绍一下，为什么选择它？

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

3. 运行内核、业务扩展和服务入口如何分层？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

4. Durable Checkpoint 具体保存了什么？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

5. 这部分是自行实现的，还是直接使用 LangGraph 的快照？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

6. 为什么只用 LangGraph 承载核心循环，而不是把全部运行机制交给它？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

7. 工具调用怎样保证幂等？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

8. 追问：调用时连接断了，但不知道外部操作是否成功，这种情况怎么办？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

9. 幂等和可靠性由 Harness 保证，还是由具体业务工具保证？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

10. 运行租约和 Fencing Token 分别解决什么问题？

答：任务领取在事务内用 status=pending 与 attempts 条件更新，只允许成功者执行；每次领取生成随机 UUID lease_token，并增加 attempt。写入及完成时校验 token、generation、状态和租约未过期，拒绝旧执行者。这里 UUID 是相等性凭证，并非单调 fencing token；若对外部存储做 fencing，需要权威存储发放单调版本，且外部接收端比较版本拒绝旧写入。

依据 / 对照：backend/app/infrastructure/repositories/jobs.py；backend/app/jobs/context.py

11. Token 为什么需要单调递增，由谁发放？

答：任务领取在事务内用 status=pending 与 attempts 条件更新，只允许成功者执行；每次领取生成随机 UUID lease_token，并增加 attempt。写入及完成时校验 token、generation、状态和租约未过期，拒绝旧执行者。这里 UUID 是相等性凭证，并非单调 fencing token；若对外部存储做 fencing，需要权威存储发放单调版本，且外部接收端比较版本拒绝旧写入。

依据 / 对照：backend/app/infrastructure/repositories/jobs.py；backend/app/jobs/context.py

12. Worker 写状态时怎样校验 Token，是否使用 CAS？

答：任务领取在事务内用 status=pending 与 attempts 条件更新，只允许成功者执行；每次领取生成随机 UUID lease_token，并增加 attempt。写入及完成时校验 token、generation、状态和租约未过期，拒绝旧执行者。这里 UUID 是相等性凭证，并非单调 fencing token；若对外部存储做 fencing，需要权威存储发放单调版本，且外部接收端比较版本拒绝旧写入。

依据 / 对照：backend/app/infrastructure/repositories/jobs.py；backend/app/jobs/context.py

13. 平台的节点抽象是什么样的？

答：当前节点是接受 State、返回局部状态更新的 Python 函数，StateGraph 负责边与条件路由；guarded_node 包装预算、安全、锁和血缘捕获。不是用户可配置节点插件，也没有统一画布节点 DSL。现有聊天节点包括解析、草稿校验、营养准备、保存、追问及回复。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

14. 举一个比较复杂的节点，说明它如何执行。

答：可讲 prepare_nutrition：读取草稿中的食物、分量和已知营养，识别用户实际总量/每100g标签，缺失字段按授权粗估，再做范围、基本一致性、来源和指纹校验，结果进入 prepared_events。分量变更会使旧估算失效；失败保持未知，后续保存路径共用准备结果，重放不应重复估算。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

15. 业务数据、逻辑和动作如何接入 Workflow 或 Agent？

答：工程使用健康事件、结构化事实和派生结果，没有知识图谱或企业本体平台。扩展可由业务域定义实体、关系、动作和授权语义，运行层只接收版本化查询/动作接口；图查询参数化并限制跳数、返回量和租户范围，结果带实体/来源标识。不能把模型生成的关系直接当作权威事实入库。

依据 / 对照：backend/app/contracts/；backend/app/health/；后半为设计

16. 多个任务共享同一张执行图时，运行状态怎样隔离？

答：编译图可以共享，运行状态由 thread_id 隔离。工程用 request/workflow/conversation 键控锁和 Engine 短临界区协调本地并发；模型 I/O 时释放总锁，返回后核对来源版本、epoch 和 workflow generation，避免旧结果写回。同数据目录还持有 OS 独占锁；这不是多个 Engine 的分布式并发方案。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/infrastructure/data_owner.py；backend/tests/test_workflow_concurrency.py

17. 动态 DSL 每次编译为图，会不会影响性能？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

18. Agent 如何持续做评测？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

19. 测试用例、参考轨迹、任务完成度和 Judge 怎么配合？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

20. 如果主 Agent 跳过了必须执行的规则检查，却直接完成发布，怎么发现和阻止？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

21. 如何声明必须调用的工具、必须生成的产物？

答：设计上把 required_checks、required_tools、required_artifacts 与版本、依赖和验收规则存到服务端任务契约，不能由模型自行删除；终态提交前校验实际执行回执和产物。本项目用固定图边、prepared_record 校验与 activate_plan 前检查表达这类强制约束，没有通用声明式完成契约。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

22. 最后的校验没通过时怎么处理？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

23. 将 Workflow 封装为 Skill，会不会限制 Agent 自由度，又带来什么成本？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

24. 大量运行中任务需要迁移时，怎样处理？

答：当前未实现大规模在途任务迁移。设计上先冻结接收新任务、排空或持久化等待点，按任务保存图/契约版本、checkpoint 与外部操作 ID；新执行者领取新租约后续跑，旧执行者失去写权限。无法兼容的状态需显式迁移或保留旧版本跑完，不能直接用新图解释旧状态。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

25. 追问：如何避免任务丢失或一直卡住？

答：工程持久化任务状态、租约、attempt 与 next_attempt_at，重启回收过期领取，有限重试后进入 failed，并提供 jobs/status 和离线 retry。过期提醒 skipped，删除相关任务 cancelled；失败不能无限 pending。独立 Worker 和大规模任务迁移仍未实现。

依据 / 对照：backend/app/jobs/planner.py；backend/app/jobs/runner.py；backend/app/jobs/handlers.py

26. 如果同时有独立复现和团队项目，如何区分各自覆盖的能力与个人贡献？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

27. 如何比较 LangGraph、AgentScope、Google ADK 等框架？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

五、美团 · 一面

1. 选一个最有代表性的项目，说明解决的问题、业务价值和个人职责。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 为什么划分为通用运行内核、业务扩展和服务入口？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

3. 业务扩展模块与 Skill 是什么关系？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

4. 除了声明文件或 YAML 配置，还包含哪些真正的运行逻辑？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

5. 用代码实现的 Workflow，怎样装配进运行时？

答：当前 Engine 初始化时调用 build_chat/build_plan/build_analysis，将 Python 节点及边 compile(checkpointer=...) 后缓存为图对象。图代码随服务发布；不是把 YAML 动态导入即执行。未来装配可按版本注册工厂，校验输入输出契约并明确旧 checkpoint 的兼容策略。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

6. Tool 和 Skill 数量增加后，上下文怎么控制？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

7. 不同业务域的 Sub-agent，能力和权限怎样划分？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

8. 渐进式披露具体怎样实现？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

9. 这些设计与标准 ReAct Loop 或现有开源 Agent 有什么不同？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

10. 你主要负责运行稳定性，还是也负责最终输出质量？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

11. Workflow 什么时候接管执行，执行后如何归还控制权？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

12. 触发条件是固定规则，还是由模型决定？

答：两者结合：模型输出结构化意图/处理计划；本地白名单、缺失字段、安全结果和计划差异决定允许走哪条固定路径。模型不能决定“授权通过”或跳过必要确认。当前不是自由选择 Skill 的 ReAct 路由器。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

13. 自然语言输入、页面操作和显式选择 Skill，是否走同一条链路？

答：自然语言走聊天解析，页面管理动作走明确 API，计划确认走 resume；入口不同，但保存规则、数据边界和版本校验应复用。工程没有“显式选择 Skill”入口，不能说三者已经统一到 Skill 加载链路。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

14. 必须执行某个 Skill 时，怎样保证模型不会选错或跳过？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

15. 没有调用必需工具时，系统能否发现和补救？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

16. 举一个真实 Bad Case，说明原因、修复过程和结果。

答：代码与测试体现的一个问题是：“帮我记录昨天吃得有点咸”缺少具体食物，不能写空餐食，也不能在后续补充时丢掉原日期。方案是先提取允许不完整的草稿，本地判断必填字段，interrupt 持久化待补项，恢复仅 patch 当前字段，并用日期/证据/重放测试验收。这是可从仓库复现的技术案例，不据此虚构个人线上故障或收益。

依据 / 对照：backend/tests/test_draft_clarification.py；backend/tests/test_contextual_replies.py；backend/app/workflows/chat_flow.py

17. 追问：除了修改 Prompt，还采取了什么工程措施？

答：代码与测试体现的一个问题是：“帮我记录昨天吃得有点咸”缺少具体食物，不能写空餐食，也不能在后续补充时丢掉原日期。方案是先提取允许不完整的草稿，本地判断必填字段，interrupt 持久化待补项，恢复仅 patch 当前字段，并用日期/证据/重放测试验收。这是可从仓库复现的技术案例，不据此虚构个人线上故障或收益。

依据 / 对照：backend/tests/test_draft_clarification.py；backend/tests/test_contextual_replies.py；backend/app/workflows/chat_flow.py

18. 跨多轮对话使用同一个 Skill，会不会反复注入相同内容？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

19. 哪些消息会持久化到历史记录里？

答：运行历史和业务事实分开：草稿/待补字段进 checkpoint，已校验记录进事件账本，显式偏好进长期事实。日志不保存健康原文、prompt 或模型响应。处理前应明确哪些内容进入持久业务，哪些只在当前流程使用；本项目并不默认把历史助手消息发回模型，过期与删除还需清理 checkpoint 内原文副本。

依据 / 对照：backend/app/workflows/state.py；backend/app/observability/；docs/四层记忆实现说明.md

20. 追问：不希望保存的内容在哪一步过滤，数据模型怎样区分？

答：运行历史和业务事实分开：草稿/待补字段进 checkpoint，已校验记录进事件账本，显式偏好进长期事实。日志不保存健康原文、prompt 或模型响应。处理前应明确哪些内容进入持久业务，哪些只在当前流程使用；本项目并不默认把历史助手消息发回模型，过期与删除还需清理 checkpoint 内原文副本。

依据 / 对照：backend/app/workflows/state.py；backend/app/observability/；docs/四层记忆实现说明.md

21. 创建一个结果与修改已有结果，是否分别处理？

答：本项目的产物主要是记录卡片、计划、复盘和导出数据，没有通用文档布局编辑器。创建与修改分别处理：修改先绑定目标和版本，校验字段、权限与来源，保留修订。超出支持字段或能力时明确说明并给可操作替代，不能生成一段“已修改”的文本冒充保存；普通回复也不能改变业务状态。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/contracts/；backend/app/policy/output.py

22. 用户提出超出能力范围的请求时，系统怎么回应？

答：本项目的产物主要是记录卡片、计划、复盘和导出数据，没有通用文档布局编辑器。创建与修改分别处理：修改先绑定目标和版本，校验字段、权限与来源，保留修订。超出支持字段或能力时明确说明并给可操作替代，不能生成一段“已修改”的文本冒充保存；普通回复也不能改变业务状态。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/contracts/；backend/app/policy/output.py

23. 普通文本回答与结构化 Artifact 如何区分？

答：本项目的产物主要是记录卡片、计划、复盘和导出数据，没有通用文档布局编辑器。创建与修改分别处理：修改先绑定目标和版本，校验字段、权限与来源，保留修订。超出支持字段或能力时明确说明并给可操作替代，不能生成一段“已修改”的文本冒充保存；普通回复也不能改变业务状态。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/contracts/；backend/app/policy/output.py

24. 如果用户要求修改 Artifact 中不支持的字段或布局，怎么办？

答：本项目的产物主要是记录卡片、计划、复盘和导出数据，没有通用文档布局编辑器。创建与修改分别处理：修改先绑定目标和版本，校验字段、权限与来源，保留修订。超出支持字段或能力时明确说明并给可操作替代，不能生成一段“已修改”的文本冒充保存；普通回复也不能改变业务状态。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/contracts/；backend/app/policy/output.py

25. 多租户隔离在哪些层实现？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

26. 沙箱解决什么问题？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

27. 工具幂等与分布式锁是什么关系？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

28. 超时、重试、降级和错误返回怎样处理？

答：错误区分输入/权限/冲突、模型上游、预算和运行故障，HTTP 层输出固定错误码和关联信息。可重试失败要有次数与退避；参数错或权限拒绝不能盲重试。模型解析失败明确报错；营养估算失败保存营养未知；表达选择失败使用本地合法选项。三类降级语义不同，不能把失败偷偷改成演示成功。

依据 / 对照：backend/app/common/errors.py；backend/app/modeling/errors.py；backend/app/modeling/transport.py；backend/tests/test_error_contract.py

29. 每次任务是否都有独立工作空间？

答：没有每任务独立文件工作空间；当前是单用户共享数据目录，通过 thread、事件 ID、版本和锁区分工作流。若要执行不可信脚本，必须另建受控目录和沙箱并限制挂载，不能把共享目录模型描述成安全的租户隔离。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

30. Agent 从一个业务域访问另一个业务域时，权限怎样控制？

答：模型不能提交任意 SQL、字段或记录 ID；本地先绑定目标、批准固定视图，再用 ModelBoundary 发一次性授权，消费前累计计数。processing_plan 仅接收当前输入和操作状态，后续任务按需读取最少数据；local_only 禁止全部外发且随工作流恢复保留。当前没有多用户身份体系，不能把数据最小化等同于多租户鉴权。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/modeling/payloads.py；docs/模型数据访问边界.md

31. 当前实际使用规模如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

32. 为什么不直接用通用 Agent，加行业专家和同一套 MCP 工具？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

33. 追问：在相同模型、工具和任务条件下，自研方案到底好在哪里？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

34. 如果通用 Agent 效果更好，客户为什么还要为自研方案付费？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

35. 怎样避免最终变成高成本的定制开发？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

36. 探索型任务和稳定的生产任务，是否应该采用不同方案？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

37. 产品或解决方案角色如何参与定义价值，而不是只由研发判断？

答：由产品/领域人员定义可验收任务、失败成本和不支持范围，研发给可观察且可验证的结果，双方用同一组用例比较方案与交付成本。本工程可用完整记录、追问完成、纠错和安全边界作为产品验收；没有客户价值访谈、付费或真实收益数据，需另行补证。

六、拼多多 · 一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 项目是否已经落地，实际使用规模如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

3. 怎样区分 Agent Harness 与 Agent Framework？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

4. 运行内核包含哪些主要模块？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

5. 项目主要面向 ToB 还是 ToC？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

6. 通用运行层与业务能力如何分层？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

7. 多个任务共享 Worker 时，状态怎么管理？

答：编译图可以共享，运行状态由 thread_id 隔离。工程用 request/workflow/conversation 键控锁和 Engine 短临界区协调本地并发；模型 I/O 时释放总锁，返回后核对来源版本、epoch 和 workflow generation，避免旧结果写回。同数据目录还持有 OS 独占锁；这不是多个 Engine 的分布式并发方案。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/infrastructure/data_owner.py；backend/tests/test_workflow_concurrency.py

8. 为什么同一时刻只激活一个 Skill？

答：本项目没有单 Skill 激活机制。设计上单一活动 Skill 可减少上下文污染、规则冲突和恢复歧义，但会限制真正可并行任务；是否采用取决于任务依赖，不是通用定律。若支持多活动 Skill，需独立作用域、预算与明确的控制权汇聚。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

9. Workflow 与 ReAct 如何衔接？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

10. 不同行业的业务差异怎样处理，能否不修改内核？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

11. 共用工作空间会不会导致文件或数据串到其他任务？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

12. 沙箱如何设计？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

13. Docker、Kubernetes 或 Serverless 分别适合什么情况？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

14. 文件、网络和系统权限如何限制？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

15. Skill 选择正确与否，如何评估？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

16. 主 Agent、专用 Agent 和 Sub-agent 如何配置？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

17. Memory 由谁决定写入，什么时候写？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

18. 记忆存在哪里，怎样向量化和检索？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

19. 记忆只是不断追加吗，怎样更新和删除？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

20. 新旧记忆发生冲突怎么办？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

21. 低价值记忆怎样遗忘？

答：待补和待确认上下文默认闲置 7 天过期，完成上下文保留 30 天；事件超过一年按完整过期月份归档。隐私删除先预览影响范围，再以持久化作业清理来源关联、共享原文、回执与 checkpoint，保留最小墓碑阻止旧请求重建。独立健康事实不能只因共享一句原文就无差别删掉；血缘不足时明确扩大范围并重新确认。

依据 / 对照：docs/精准删除与数据血缘解决方案.md；backend/app/privacy/；backend/tests/test_privacy_deletion.py

22. 追问：如果存进去的是错误记忆，怎样纠正？

答：长期事实有版本、期限、陈述状态和替代关系。明确长期改口保留历史并替代旧偏好；临时偏好不静默覆盖长期事实；过敏/专业限制不能被普通喜好解除，期限到后需核对。纠错先绑定原记录并校验版本，再保存修订、使相关派生结果待更新。记忆不能覆盖执行权限、安全边界或必需审批。

依据 / 对照：backend/app/health/fact_resolution.py；backend/app/infrastructure/repositories/facts.py；backend/tests/test_structured_facts.py

23. 工作记忆、短期记忆和长期记忆如何区分？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

24. 多 Agent 相互交接时，会不会出现循环调用？

答：会。扩展方案应持久化委派链，限制深度、总次数/时间预算，按目标与 task_id 去重；子任务不随意再回交相同目标，父任务显式收敛结果。交接失败或无进展应结束为澄清/失败。本工程没有通用多 Agent handoff。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

25. Handoff 应传哪些结构化信息？

答：扩展设计应持久化 parent_task_id、child_task_id、目标、权限、输入快照、依赖、状态、结果引用及 continuation。交接不传完整对话；只传已确认事实、剩余任务、失败与预算。恢复先读取依赖图和子任务结果，复用已完成结果，再唤醒父任务；对子任务创建和父任务汇聚都用幂等键。本项目 outbox 可示范可靠创建后续任务，但不等同于通用多 Agent。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

26. 进程重启后，Checkpoint 如何恢复？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

27. 追问：父任务和子任务存在依赖时，怎样恢复它们的关系和执行顺序？

答：扩展设计应持久化 parent_task_id、child_task_id、目标、权限、输入快照、依赖、状态、结果引用及 continuation。交接不传完整对话；只传已确认事实、剩余任务、失败与预算。恢复先读取依赖图和子任务结果，复用已完成结果，再唤醒父任务；对子任务创建和父任务汇聚都用幂等键。本项目 outbox 可示范可靠创建后续任务，但不等同于通用多 Agent。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

28. 工具已经产生副作用，恢复后如何避免重复操作？

答：工程通过 request_id + payload 指纹返回旧回执，事件写入使用 thread_id:event:index，恢复请求也绑定中断点。锁减少并发冲突，唯一键和持久回执负责重放去重。外部服务若已执行但本地未保存结果，需要同一个外部幂等键或查询外部操作状态；不支持这些能力时应保留 unknown 并对账，不能盲重试。模型调用仍可能在远端接受后、本地落盘前重复。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py；backend/tests/test_runtime_recovery.py

29. 完整 RAG 链路如何设计？

答：项目对健康记录使用 SQLite 固定视图查询，没有文档上传知识库、Embedding、向量数据库或 Reranker。扩展方案是解析保留层级/来源，切分建索引，权限过滤后召回、融合、重排，按预算组装证据再回答。小规模明确日期/类型/关键词查询先用结构化检索，不能为了面试把事件检索说成已上线混合 RAG。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

30. 如果记忆规模很小，还有必要使用向量库和 Reranker 吗？

答：当前健康事实有日期、类型与版本，SQLite 查询更便宜且可解释。grep/glob 适合文件名、标识符和精确文本，向量适合语义近似，层级索引适合有稳定章节结构的文档。它们没有绝对优劣，应按语料规模、更新频率、权限、证据定位和延迟评测；少量记忆不必先上向量库和重排。

依据 / 对照：backend/app/infrastructure/repositories/events.py；后半为设计

31. 怎样看待 grep、glob、PageIndex 等不同检索方式？

答：当前健康事实有日期、类型与版本，SQLite 查询更便宜且可解释。grep/glob 适合文件名、标识符和精确文本，向量适合语义近似，层级索引适合有稳定章节结构的文档。它们没有绝对优劣，应按语料规模、更新频率、权限、证据定位和延迟评测；少量记忆不必先上向量库和重排。

依据 / 对照：backend/app/infrastructure/repositories/events.py；后半为设计

32. 网页内容中带有 Prompt Injection，怎么办？

答：把用户输入、检索文本和模型结果视为数据；执行层只接受固定 schema 和动作白名单，重新检查授权、版本与业务规则。工程在解析、估算、保存、回复和计划激活前做安全检查，输出日期、数值和保存状态由本地绑定。即使模型说“跳过审批”也无权激活计划；Prompt 禁令只是补充，不能替代执行检查。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/workflows/safety.py；backend/app/policy/output.py

33. 信任边界、最小权限和人工确认应如何配合？

答：把用户输入、检索文本和模型结果视为数据；执行层只接受固定 schema 和动作白名单，重新检查授权、版本与业务规则。工程在解析、估算、保存、回复和计划激活前做安全检查，输出日期、数值和保存状态由本地绑定。即使模型说“跳过审批”也无权激活计划；Prompt 禁令只是补充，不能替代执行检查。

依据 / 对照：backend/app/workflows/concurrency.py；backend/app/workflows/safety.py；backend/app/policy/output.py

34. Thread.sleep(0) 有什么作用？

答：Java 的 sleep(0) 不提供可靠调度、公平性或跨线程可见性保证，也不释放已持有 monitor；不能用它等待某状态变更。线程协作应使用锁、条件、阻塞队列或其他并发原语；是否让出处理器属于实现与调度行为，不能当作业务正确性的前提。项目后端是 Python，此为语言基础题。

依据 / 对照：Java Language Specification 17.3

35. 【算法题】 给一个含重复元素的数组，重新排列，使相邻元素不相同。

答：最大频次超过 ceil(n/2) 时无解；否则用最大堆每次取频次最多且不等于上一项的元素，并将上一项延迟放回堆。时间 O(n log k)、空间 O(k)，k 是不同元素数。附录提供实现与可行/不可行边界。

36. 【算法题·追问】 什么情况下无解，如何判断？

答：最大频次超过 ceil(n/2) 时无解；否则用最大堆每次取频次最多且不等于上一项的元素，并将上一项延迟放回堆。时间 O(n log k)、空间 O(k)，k 是不同元素数。附录提供实现与可行/不可行边界。

37. 平时有什么兴趣，最近读了什么书？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

七、腾讯 · 一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 项目有哪些模块，一次请求的执行流程是什么？

答：小程序提交 request_id 和处理模式；服务端检查回执与指纹，定位或创建 thread；安全入口先判定允许能力；模型解析草稿，本地校验缺失字段与权限；营养准备后保存完整独立记录，缺项则 interrupt 追问；最终本地渲染结果并保存回执。恢复时校验 thread、interrupt_id 和请求指纹，再继续原图。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py

3. 系统怎样部署，最终由谁使用？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

4. 上下文预算如何分配？

答：工程优先不发送无关内容：首轮不带最近聊天和健康历史，后续只发经批准的固定视图。咨询饮食汇总最多近 7 天，体重/分量上下文最多近 28 天；明确更短范围就按更短范围。关键日期、数值、来源、未知状态和 local_only 必须保留。当前有字节和调用预算，未实现按 tokenizer 的动态 Token 分配或模型摘要压缩。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

5. 历史消息和工具结果分别如何压缩？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

6. 压缩丢掉的信息后续又需要了，怎么找回来？

答：当前主要用字段投影、条数/时间窗限制控制上下文，没有已实现的动态摘要压缩器。扩展时先裁无关项，再保留任务约束、当前状态、用户决定、来源与错误；长工具结果保存原件引用，摘要带版本与可回查定位。压缩不能当作删除，取回也必须重新授权；一年归档后的逐餐细节不可恢复，不能承诺摘要可还原全部原文。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

7. 在什么时候触发压缩？同步做还是异步做？

答：本工程没有自动摘要压缩触发器，采用固定字段/条数/时间窗和发送字节上限。若扩展，应在请求前估算输入加输出预留预算，超阈值同步做确定性裁剪；耗时摘要可异步预生成，但必须绑定来源版本，不能使用过期摘要。隐私授权、安全约束与未完成任务信息不得压掉。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/health/retention.py；后半为设计

8. 是否支持多模态输入，整体链路如何处理？

答：当前入口主要是文本记录与页面表单，没有图像、语音、视频或文档问答能力。扩展需要在入口校验 MIME/大小/权限，识别任务和数据授权后选择视觉/OCR或文本处理器，保存来源与解析置信信息；图片、表格不能脱离页/单元格坐标。路由由本地能力策略约束，不允许模型自行外发整份健康材料。

依据 / 对照：miniprogram/src/api.ts；backend/app/api/；后半为设计

9. 视觉模型与文本模型什么时候切换，由谁决定？

答：当前入口主要是文本记录与页面表单，没有图像、语音、视频或文档问答能力。扩展需要在入口校验 MIME/大小/权限，识别任务和数据授权后选择视觉/OCR或文本处理器，保存来源与解析置信信息；图片、表格不能脱离页/单元格坐标。路由由本地能力策略约束，不允许模型自行外发整份健康材料。

依据 / 对照：miniprogram/src/api.ts；backend/app/api/；后半为设计

10. PDF、Word 等文档怎么处理？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

11. 历史数据与实时信息如何结合，更新怎样生效？

答：当前不是流式事件系统。健康历史由已保存事件与修订构成，页面通过 API 拉取，变更由版本与状态刷新生效；模型 I/O 返回后再核对读到的数据是否已变。若题目指 SSE 历史补发，需另建持久事件日志和游标，不能把业务历史查询等同于消息续传。

依据 / 对照：当前未实现；参照 backend/app/workflows/runtime.py 的业务状态恢复

12. 用户手工编排 Workflow，与自然语言生成计划有什么区别？

答：手工编排约束由人确定，模型规划只是候选，需要静态校验、权限审核与运行验收。本项目没有手工画布或自动生成图，只有固定 Python 图和受限 processing_plan；后者不能定义任意节点、边或代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

13. DSL 如何转换为执行图？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

14. 视频生成等耗时较长的工具，怎样放进 Agent 执行流程？

答：工程未接入视频生成等外部异步工具。可扩展为：提交后持久化 external_job_id、步骤状态、幂等键和下一检查时间，然后挂起原任务；轮询或验签回调先去重并落结果，再按原 thread/step 恢复。回调不能直接越过权限和状态版本检查；超时、取消、晚到结果及服务重启都需有明确终态。

依据 / 对照：参照 backend/app/jobs/ 与 backend/app/workflows/runtime.py；未实现该集成

15. 计划保存在哪里？步骤失败、中断后怎样继续？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

16. 异步结果通过轮询还是回调拿到？

答：工程未接入视频生成等外部异步工具。可扩展为：提交后持久化 external_job_id、步骤状态、幂等键和下一检查时间，然后挂起原任务；轮询或验签回调先去重并落结果，再按原 thread/step 恢复。回调不能直接越过权限和状态版本检查；超时、取消、晚到结果及服务重启都需有明确终态。

依据 / 对照：参照 backend/app/jobs/ 与 backend/app/workflows/runtime.py；未实现该集成

17. 追问：拿到结果后，怎样恢复原来的任务？

答：工程未接入视频生成等外部异步工具。可扩展为：提交后持久化 external_job_id、步骤状态、幂等键和下一检查时间，然后挂起原任务；轮询或验签回调先去重并落结果，再按原 thread/step 恢复。回调不能直接越过权限和状态版本检查；超时、取消、晚到结果及服务重启都需有明确终态。

依据 / 对照：参照 backend/app/jobs/ 与 backend/app/workflows/runtime.py；未实现该集成

18. Human-in-the-loop 在什么时候介入？

答：记录缺必填字段时 interrupt；计划首次或单次/累计超过阈值时等待确认。恢复请求必须匹配 thread_id、interrupt_id、request_id 与指纹，防止旧回答消费新中断。补充只改待补字段，保留日期与已知内容；计划确认还检查资料、基线与限制是否变化。拒绝、取消、过期都是独立状态，不能当作执行成功。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/planning.py；backend/tests/test_draft_clarification.py

19. 用户补充输入、修改方向和等待确认，分别怎么处理？

答：记录缺必填字段时 interrupt；计划首次或单次/累计超过阈值时等待确认。恢复请求必须匹配 thread_id、interrupt_id、request_id 与指纹，防止旧回答消费新中断。补充只改待补字段，保留日期与已知内容；计划确认还检查资料、基线与限制是否变化。拒绝、取消、过期都是独立状态，不能当作执行成功。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/planning.py；backend/tests/test_draft_clarification.py

20. 恢复执行时，如何确定输入对应哪个任务、哪个等待点？

答：记录缺必填字段时 interrupt；计划首次或单次/累计超过阈值时等待确认。恢复请求必须匹配 thread_id、interrupt_id、request_id 与指纹，防止旧回答消费新中断。补充只改待补字段，保留日期与已知内容；计划确认还检查资料、基线与限制是否变化。拒绝、取消、过期都是独立状态，不能当作执行成功。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/planning.py；backend/tests/test_draft_clarification.py

21. RAG 链路如何设计？

答：项目对健康记录使用 SQLite 固定视图查询，没有文档上传知识库、Embedding、向量数据库或 Reranker。扩展方案是解析保留层级/来源，切分建索引，权限过滤后召回、融合、重排，按预算组装证据再回答。小规模明确日期/类型/关键词查询先用结构化检索，不能为了面试把事件检索说成已上线混合 RAG。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

22. Excel 或大型表格怎样解析和检索？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

23. 【算法题】 二叉树的层序遍历。题型来自面试后的补充回忆，完整题面及作答过程未能确认。

答：用 deque 做 BFS：每轮固定当前队列长度，只弹出这一层并把子节点入队，形成二维结果。时间 O(n)、辅助空间 O(w)，w 是最大层宽。原 PDF 题面未确认，附录只给经典层序遍历版本。

八、蚂蚁 · 一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 如果参与过 Java 微服务项目，主要负责哪些模块？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

3. 以实时通信系统为例，底层采用 SIP、VoIP 还是其他协议，应如何选择？

答：本项目没有坐席、SIP 或 VoIP 组件。设计时区分 SIP 等信令控制与媒体承载，通信组件维护权威呼叫状态，上层编排业务；事件需带 call_id、序号/版本和幂等键，允许重放并定期对账。协议选择取决于互通、网络、媒体与终端要求，不能从健康 Agent 仓库推断已有通信项目经验。

4. 通信控制层与上层业务服务分别负责什么？

答：本项目没有坐席、SIP 或 VoIP 组件。设计时区分 SIP 等信令控制与媒体承载，通信组件维护权威呼叫状态，上层编排业务；事件需带 call_id、序号/版本和幂等键，允许重放并定期对账。协议选择取决于互通、网络、媒体与终端要求，不能从健康 Agent 仓库推断已有通信项目经验。

5. 系统的坐席和并发规模如何？

答：本项目没有坐席、SIP 或 VoIP 组件。设计时区分 SIP 等信令控制与媒体承载，通信组件维护权威呼叫状态，上层编排业务；事件需带 call_id、序号/版本和幂等键，允许重放并定期对账。协议选择取决于互通、网络、媒体与终端要求，不能从健康 Agent 仓库推断已有通信项目经验。

6. 连接状态由底层通信组件还是上层调度服务维护？

答：本项目没有坐席、SIP 或 VoIP 组件。设计时区分 SIP 等信令控制与媒体承载，通信组件维护权威呼叫状态，上层编排业务；事件需带 call_id、序号/版本和幂等键，允许重放并定期对账。协议选择取决于互通、网络、媒体与终端要求，不能从健康 Agent 仓库推断已有通信项目经验。

7. 怎样保证通话状态同步和准确？

答：本项目没有坐席、SIP 或 VoIP 组件。设计时区分 SIP 等信令控制与媒体承载，通信组件维护权威呼叫状态，上层编排业务；事件需带 call_id、序号/版本和幂等键，允许重放并定期对账。协议选择取决于互通、网络、媒体与终端要求，不能从健康 Agent 仓库推断已有通信项目经验。

8. 状态事件通过消息队列还是回调传递？

答：本项目没有坐席、SIP 或 VoIP 组件。设计时区分 SIP 等信令控制与媒体承载，通信组件维护权威呼叫状态，上层编排业务；事件需带 call_id、序号/版本和幂等键，允许重放并定期对账。协议选择取决于互通、网络、媒体与终端要求，不能从健康 Agent 仓库推断已有通信项目经验。

9. 如果参与过数据处理平台，平台解决什么问题，个人负责哪部分？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

10. 以设备数据为例，怎样设计上报和存储链路？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

11. 使用什么时序数据库？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

12. 如果数据上报丢失，怎样发现和定位？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

13. 用户怎样配置指标计算逻辑？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

14. 流式数据的时间窗口如何实现？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

15. 追问：为什么没有采用滑动窗口？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

16. 为什么需要中间汇总计算层？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

17. 指标定义发生变化后，历史数据怎么重算？

答：本项目派生结果记录方法版本、窗口、生成时间和来源版本，源变化会提示更新/复核；已确认目标不会自动覆盖，历史复盘保留原窗口。重算应固定输入快照与算法版本，以幂等任务生成新版本再切换；超过一年只有月摘要时不能还原逐餐重算。若改用实时流系统，还应保留可重放原始事件及去重键。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/derived.py

18. 追问：如果重新设计指标版本与重算机制，会怎样改进？

答：本项目派生结果记录方法版本、窗口、生成时间和来源版本，源变化会提示更新/复核；已确认目标不会自动覆盖，历史复盘保留原窗口。重算应固定输入快照与算法版本，以幂等任务生成新版本再切换；超过一年只有月摘要时不能还原逐餐重算。若改用实时流系统，还应保留可重放原始事件及去重键。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/derived.py

19. 是否考虑过使用 Redis 承载中间计算？

答：项目没有 Kafka、Redis 流处理或专用时序数据库，健康记录使用 SQLite。设备上报可设计 event_id、设备序列、事件/接收时间和 schema 版本，接入鉴权校验后持久化，再异步计算；幂等写入与序号缺口/对账发现丢失。按业务选滚动、滑动或会话窗口，明确水位线、迟到修正和重算规则，中间汇总用于降低重复扫描而非掩盖数据丢失。

20. 为什么选择 LangGraph？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

21. LangGraph 在平台中具体承担什么职责？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

22. 为什么不用封装程度更高的 Agent 框架？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

23. 从文件上传到问答，知识库的完整处理链路是什么？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

24. 超大型文件或十几万行的 Excel 如何处理？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

25. 关联信息在文档中相距很远，怎样保证一起召回？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

26. Metadata Filter 在检索中怎么使用？

答：Metadata Filter 应在召回前或检索引擎内部限制租户、文档版本、时间和访问范围，再做相似检索；仅在 TopK 后过滤可能丢失正确候选，且不可作为唯一权限边界。本项目日期/类型由固定 SQLite 查询约束，没有向量检索的 metadata filter。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

27. PPL 语义边界检测如何实现？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

28. 追问：这种切分方案有哪些问题？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

29. 如果 Recall@5 突然大幅下降，怎么排查？

答：项目没有相关线上事故，可按设计回答：先固定失败 query 和金标准，确认指标/评测集未变，再核对语料版本、过滤条件、索引刷新、Embedding 版本/维度/归一化和召回参数；分阶段检查正确证据在哪一步丢失。相同文本也不保证不同服务版本给出相同向量，应固定模型/预处理，多次比较向量与排名，Rerank 同样检查格式、完整性及排序抖动。

30. 追问：用户、场景和配置都没有变化，应优先怀疑什么？

答：项目没有相关线上事故，可按设计回答：先固定失败 query 和金标准，确认指标/评测集未变，再核对语料版本、过滤条件、索引刷新、Embedding 版本/维度/归一化和召回参数；分阶段检查正确证据在哪一步丢失。相同文本也不保证不同服务版本给出相同向量，应固定模型/预处理，多次比较向量与排名，Rerank 同样检查格式、完整性及排序抖动。

31. Embedding 是否“幂等”，同一输入能否得到稳定输出？

答：项目没有相关线上事故，可按设计回答：先固定失败 query 和金标准，确认指标/评测集未变，再核对语料版本、过滤条件、索引刷新、Embedding 版本/维度/归一化和召回参数；分阶段检查正确证据在哪一步丢失。相同文本也不保证不同服务版本给出相同向量，应固定模型/预处理，多次比较向量与排名，Rerank 同样检查格式、完整性及排序抖动。

32. 追问：怎样验证输出的稳定性？

答：项目没有相关线上事故，可按设计回答：先固定失败 query 和金标准，确认指标/评测集未变，再核对语料版本、过滤条件、索引刷新、Embedding 版本/维度/归一化和召回参数；分阶段检查正确证据在哪一步丢失。相同文本也不保证不同服务版本给出相同向量，应固定模型/预处理，多次比较向量与排名，Rerank 同样检查格式、完整性及排序抖动。

33. 使用什么 Reranker？

答：本项目没有 Reranker，不能虚构模型型号。若选型，应在固定候选集比较排序质量、延迟、成本、语种和最大长度，检查证据覆盖与截断影响；重排器不能生成新事实或扩大访问范围。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

34. Reranker 的输出是否稳定？

答：项目没有相关线上事故，可按设计回答：先固定失败 query 和金标准，确认指标/评测集未变，再核对语料版本、过滤条件、索引刷新、Embedding 版本/维度/归一化和召回参数；分阶段检查正确证据在哪一步丢失。相同文本也不保证不同服务版本给出相同向量，应固定模型/预处理，多次比较向量与排名，Rerank 同样检查格式、完整性及排序抖动。

35. 追问：如何保证模型排序结果符合格式和行为约束？

答：结构上校验候选 ID 是否属于输入集合、是否重复/遗漏、分数是否合法、是否含额外正文；行为上用排序标注、重复试验和越权输入回归检查。格式合法不代表相关性正确；对异常结果应回退原候选顺序或拒答。工程输出选择器已有类似 ID 白名单思想，但未实现 Reranker。

36. Agent 执行链路如何做到可观测？

答：运行出口只存固定事件、耗时、结果及关联 ID，不记健康原文、prompt、模型响应、SQL 参数或密钥。日志有轮转/保留，图执行关闭第三方内容 tracing；内部 metrics 默认关闭。质量账本随业务事务去重和删除；运行计数重启会重置，异步日志硬退出可能丢失。当前未接外部遥测平台或按角色的日志查看授权。

依据 / 对照：docs/可观测性与健康数据保护开发方案.md；backend/app/observability/

37. 追问：有了可观测性，怎样进一步做到可评测？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

38. 最终输出与工具调用轨迹分别怎样评价？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

39. 为什么独立建设 Agent Harness？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

40. 通用运行内核、服务入口与业务扩展怎样分工？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

41. 如何约束业务实现，避免侵入运行内核？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

42. Codex 与 Claude Code 有什么区别？

答：该仓库不能证明个人使用这两款工具的经验，也没有做同模型/同任务对照。可从模型、上下文选择、工具协议、权限、恢复和交互等维度设计比较，避免把模型差异归因给 Harness；隔离因素需分别固定模型或固定执行环境做实验。本答案不做当前产品功能排名。

依据 / 对照：README.md；backend/tests/；比较方案未执行

43. 追问：行为差异主要来自模型，还是 Harness？

答：两者都可能影响。把模型、提示/上下文、工具 schema、权限、预算与任务数据分别固定或消融，才有机会判断贡献；仅比较两款完整产品的最终文本无法归因。本工程没有这样的受控实验，不能声称差异主要来自某一方。

依据 / 对照：README.md；backend/tests/；比较方案未执行

八、蚂蚁 · 二面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 业务 Agent Harness 是当前的重点工作吗？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

3. 项目团队的开发人员和其他角色怎样配置？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

4. 主要采用 AI 辅助编码，还是传统人工编码？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

5. 你在项目中具体负责什么？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

6. 多人同时使用 AI Coding 时，怎样拆分任务、控制协作和集成风险？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

7. 业务扩展模块是否按业务域划分？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

8. 运行内核由多少人维护？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

9. 你在团队中的职级和角色是什么？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

10. 不同用工类型的人员怎样分工？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

11. 运行内核、业务扩展和服务入口是独立 REST 服务，还是装配在同一个服务内？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

12. 运行内核从零实现，还是基于开源项目？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

13. LangGraph 在其中承担什么职责？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

14. 一轮 ReAct 包含哪些节点？

答：ReAct 若反复选相同工具、拿不到成功信号、失败无限重试，或压缩丢失目标就会不收敛。设计上要同时限制步骤、模型调用、总时间和输出大小，并检测无进展。工程当前是固定图，默认执行期限 120 秒、模型超时 45 秒、安全分类 8 秒，且有累计调用/字节额度；可选信息最多追问两轮，必填缺失不能为了结束而保存空记录。

依据 / 对照：backend/app/common/config.py；backend/app/common/execution_budget.py；backend/app/workflows/runtime.py

15. Observation 里记录哪些信息？

答：作为扩展设计，工具结果应含 call_id、工具版本、状态、结构化结果或错误码、来源、耗时、可重试标记及外部操作标识；长内容保存引用，仅返回摘要和定位信息。不得携带密钥或无关原文。工程已有 cards、evidence_refs、process_trace 和 model_disclosures，但没有完整通用 Observation 协议。

依据 / 对照：backend/app/workflows/state.py；backend/app/workflows/runtime.py

16. 为什么可以把这套运行内核看作最小 Harness？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

17. 业务扩展模块具体包含什么？

答：本项目按工作流、领域规则、模型边界与 Repository 分模块，但没有热插拔业务包。若平台化，扩展包应含声明、schema、执行代码、权限、迁移、测试和完成校验；新增业务通过接口装配，只有通用生命周期/持久化语义变化才改内核。契约版本、兼容性测试和灰度发布能减少并行开发相互影响，不能只靠 YAML 声明。

依据 / 对照：backend/app/workflows/engine.py；backend/app/contracts/；扩展设计

18. 追问：它与 Skill 是什么关系？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

19. MCP、外部业务系统适配和权限约束放在哪一层？

答：当前没有 MCP 或外部业务连接器。扩展时 transport/adapter 负责协议和 schema 映射，领域层实现业务语义，执行边界校验主体、scope、数据范围及副作用许可；凭据从服务端受控注入，不发给模型。Workflow 与 Agent 应共用该执行边界，而不是每个入口另写鉴权。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/modeling/payloads.py；docs/模型数据访问边界.md

20. 描述式 Skill 和 Workflow Skill 有什么区别？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

21. 确定性 Workflow 如何挂起并恢复 ReAct？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

22. 目前经过多少真实客户的验证？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

23. 为什么考虑换工作？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

24. 目前的职级、绩效和薪酬结构如何？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

25. 应用平台与 Dify、Coze、Palantir 分别是什么关系？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

26. 下层业务数据、逻辑和动作怎样被应用平台使用？

答：工程使用健康事件、结构化事实和派生结果，没有知识图谱或企业本体平台。扩展可由业务域定义实体、关系、动作和授权语义，运行层只接收版本化查询/动作接口；图查询参数化并限制跳数、返回量和租户范围，结果带实体/来源标识。不能把模型生成的关系直接当作权威事实入库。

依据 / 对照：backend/app/contracts/；backend/app/health/；后半为设计

27. 平台当前有多少客户和线上用户？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

28. 如果没有深度业务共建，平台怎样形成超越通用低代码产品的竞争力？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

29. 低代码产品本来是为了降低门槛，为什么仍需要研发人员陪同客户落地？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

30. 什么情况下需要从画布平台转向 Agent Harness？

答：本项目从一开始就是代码驱动的健康 Workflow，没有画布迁移史。设计上，当任务路径需要探索、工具选择依赖观察时，可增加受控 Agent 层；仍稳定的业务校验作为子流程复用。若流程明确，直接 Workflow 通常更容易验收，不应仅因技术趋势重写。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

31. 一次客户交付之后，哪些东西能够成为长期资产？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

32. 原平台的 Workflow 和 Agent 能否迁移到新运行时？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

33. 最复杂的 Workflow 大概有多大？

答：当前没有用户画布统计。聊天图包含安全入口与 12 个注册业务节点（parse_initial 到 respond），以及条件边和追问回路；这不是生产客户“最大流程节点数”。可展示 build_chat 的实际结构，不编造客户配置规模。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

34. 平台支持哪些节点类型？

答：当前聊天图有安全判定、解析/补丁、标准化、草稿校验、授权、营养准备、独立保存、追问准备/中断、最终保存、长期事实和回复；计划/分析也各有固定图。没有用户可配置的通用 HTTP、MCP、代码等节点库。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

35. 迁移后，原有 Workflow 能力怎样继续承载？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

36. 你说的 Workflow Skill，究竟是描述文件，还是一段可执行代码？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

37. 追问：业务逻辑是不是仍然需要研发人员编写代码？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

38. 运行内核以外，还有哪些能够稳定复用的公共资产？

答：仓库无法证明团队人数或个人编码比例。可说明方法：按模块与接口拆任务，小步提交；共享 schema 先达成契约，功能测试和故障测试一起验收，避免多人改同一状态机。可复用资产包括数据契约、模型适配、隐私边界、幂等/恢复模式和合成用例；客户特定规则单独封装。AI 辅助代码也需人工理解、审查和验证。

九、百度 · 一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 是否了解百度千帆 AppBuilder？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

3. 是否了解 Coze、Dify 等 AI 应用开发平台？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

4. 当前平台的产品定位和技术选型是什么？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

5. 为什么使用 LangGraph，平台自行实现了哪些部分？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

6. 画布式 Workflow 编排中最大的难点是什么？

答：当前图由 Python StateGraph 代码构建，没有画布 DSL 或自然语言自动生成图。扩展时先生成受限 IR，静态校验节点白名单、schema、引用、连边、终止条件和权限，再编译执行；带环图需显式预算。图定义按版本/哈希缓存，运行状态按 thread 隔离。生成后的测试必须在隔离数据上执行，修复有次数上限，不直接发布模型生成代码。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

7. 大型复杂流程如何通过子 Workflow 拆分？

答：本项目通过模块拆分代码，但没有通用子 Workflow 插件。设计上按稳定业务边界定义子流程，输入最小状态、输出结构结果，单独声明错误/中断/取消语义；父图只保存子流程 ID 与 continuation。版本和迁移必须覆盖父子状态，不能依赖调用栈恢复。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；后半为设计

8. 主流程与子流程怎样传递上下文和结果？

答：扩展设计应持久化 parent_task_id、child_task_id、目标、权限、输入快照、依赖、状态、结果引用及 continuation。交接不传完整对话；只传已确认事实、剩余任务、失败与预算。恢复先读取依赖图和子任务结果，复用已完成结果，再唤醒父任务；对子任务创建和父任务汇聚都用幂等键。本项目 outbox 可示范可靠创建后续任务，但不等同于通用多 Agent。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

9. 追问：子流程出现中断或异常，主流程如何处理？

答：扩展设计应持久化 parent_task_id、child_task_id、目标、权限、输入快照、依赖、状态、结果引用及 continuation。交接不传完整对话；只传已确认事实、剩余任务、失败与预算。恢复先读取依赖图和子任务结果，复用已完成结果，再唤醒父任务；对子任务创建和父任务汇聚都用幂等键。本项目 outbox 可示范可靠创建后续任务，但不等同于通用多 Agent。

依据 / 对照：backend/app/infrastructure/repositories/outbox.py；backend/app/jobs/runner.py

10. 代码节点怎样保证执行安全？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

11. Docker 或 Kubernetes 提供了哪些隔离机制？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

12. 从文件上传到生成回答，完整 RAG 链路是什么？

答：项目对健康记录使用 SQLite 固定视图查询，没有文档上传知识库、Embedding、向量数据库或 Reranker。扩展方案是解析保留层级/来源，切分建索引，权限过滤后召回、融合、重排，按预算组装证据再回答。小规模明确日期/类型/关键词查询先用结构化检索，不能为了面试把事件检索说成已上线混合 RAG。

依据 / 对照：README.md；backend/app/infrastructure/repositories/events.py；后半为设计

13. Excel 等结构化文档如何解析和切分？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

14. 固定切分、层级切分和语义切分如何选择？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

15. PPL 语义切分的原理和依据是什么？

答：一种做法是计算句子在前文条件下的平均负对数似然，寻找得分突变作为潜在主题边界，再结合标题、段落、最小/最大长度切分。PPL 是困惑度指标，不天然等于语义边界；受模型、tokenizer、文本长度及表格/OCR 噪声影响。阈值需在标注集调参并算推理成本。本项目未使用该方法，也没有可报的模型和收益数字。

16. 追问：怎么通过评测证明这种切分有效？

答：Recall@K 看已标注相关证据召回覆盖，MRR 看第一个相关结果的位置，NDCG 看带相关等级的排序质量；多证据问答还应检查证据集合是否齐全。回答侧单独测正确性、依据一致性、拒答和引用定位，联合记录延迟/成本。项目尚无 RAG 指标实测，不能引用安全分类测试替代检索评测。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

17. BM25、向量检索、RRF 和 Rerank 如何组合？

答：BM25 补关键词精确匹配，向量补语义召回；分别取候选，RRF 按名次融合而非直接相加不同量纲分数，再去重、用 Reranker 排相关性。MMR 在相关性与结果多样性间权衡，适合候选重复时选用；若降低证据完整性或延迟不值就不启用。本项目未使用这些组件，具体收益必须由固定数据集消融验证。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

18. 答案需要的信息跨多个 Chunk 时，怎样召回？

答：PDF 保留页码、版面、表格和 OCR 来源；PPT 保留页与图文关系；Word 按标题/段落；Excel 保留工作表、表头、类型和行键，大表用分批导入和 SQL 聚合，避免整表拼 prompt。先结构切分、再长度约束并保留父子引用；跨片证据可邻居扩展/父块回取。粒度要用同一问题集验证召回、噪声和成本；项目暂无该管线。

19. 怎样看待 Coding Agent 使用 grep 或文件遍历，而不是向量 RAG？

答：当前健康事实有日期、类型与版本，SQLite 查询更便宜且可解释。grep/glob 适合文件名、标识符和精确文本，向量适合语义近似，层级索引适合有稳定章节结构的文档。它们没有绝对优劣，应按语料规模、更新频率、权限、证据定位和延迟评测；少量记忆不必先上向量库和重排。

依据 / 对照：backend/app/infrastructure/repositories/events.py；后半为设计

20. PageIndex 一类层级索引适合什么场景？

答：当前健康事实有日期、类型与版本，SQLite 查询更便宜且可解释。grep/glob 适合文件名、标识符和精确文本，向量适合语义近似，层级索引适合有稳定章节结构的文档。它们没有绝对优劣，应按语料规模、更新频率、权限、证据定位和延迟评测；少量记忆不必先上向量库和重排。

依据 / 对照：backend/app/infrastructure/repositories/events.py；后半为设计

21. 流式节点与非流式节点如何衔接，并统一展示？

答：当前不是 SSE token 流，也没有 Redis 会话状态；页面恢复业务状态不等于恢复流式文本。若扩展，用持久事件日志保存 task_id、单调 seq/event_id、类型及版本，重连携 Last-Event-ID，只补游标后的事件，再衔接实时流；客户端按 ID 去重。快照与增量要有一致边界，历史过期时显式要求重同步，不能仅靠 Redis TTL。

依据 / 对照：当前未实现；参照 backend/app/workflows/runtime.py 的业务状态恢复

22. 追问：是否研究过 Dify 对这两类节点的抽象？

答：仓库没有对 Dify 源码阅读的证据，不能替个人作答。可分析设计：统一节点输出为状态事件、增量内容和终态结果，非流式节点在完成时发结果，流式节点发序号化增量，界面按同一任务/节点协议合并。这里只是方案，未在本项目实现。

23. 平台的客户量、调用量和并发规模怎样？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

24. 平台如何部署，Pod 和 Worker 如何组织？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

25. Python GIL 对多核并发有什么影响？

答：async 协程适合可等待 I/O，不会自动把同步阻塞函数变成非阻塞。同步端点可在线程池执行，CPU 密集任务考虑进程或独立任务服务；普通带 GIL 的 CPython 线程不能让纯 Python CPU 代码多核并行，需区分 free-threaded 构建。当前 Engine 还有锁和 SQLite 单所有者约束，单凭 FastAPI/async 不能推算商业容量。

依据 / 对照：backend/app/main.py；backend/app/workflows/concurrency.py；Python 官方 asyncio 文档

26. FastAPI 的并发依靠协程、线程还是进程？

答：async 协程适合可等待 I/O，不会自动把同步阻塞函数变成非阻塞。同步端点可在线程池执行，CPU 密集任务考虑进程或独立任务服务；普通带 GIL 的 CPython 线程不能让纯 Python CPU 代码多核并行，需区分 free-threaded 构建。当前 Engine 还有锁和 SQLite 单所有者约束，单凭 FastAPI/async 不能推算商业容量。

依据 / 对照：backend/app/main.py；backend/app/workflows/concurrency.py；Python 官方 asyncio 文档

27. 线上接口延迟突然升高，怎么排查？

答：项目没有流式 TTFT 实测。设计上客户端首个有意义内容到达时间与上游模型首 token 时间要分开计；P95 是同口径样本延迟的第 95 百分位。首响应慢先拆排队、鉴权/读取、安全分类、模型连接/推理及传输，再看锁等待、SQLite busy、上游限流和资源饱和。不能把 UI loading 或空心跳算作首个有效 token。

依据 / 对照：backend/app/observability/；后半为指标设计

28. PostgreSQL 慢查询怎么定位，EXPLAIN 和索引如何使用？

答：工程当前是 SQLite，并未接 PostgreSQL。PostgreSQL 设计题先看慢日志/pg_stat_statements，再在受控环境用 EXPLAIN (ANALYZE, BUFFERS) 区分扫描、连接、排序与 I/O；依据过滤和排序设计索引并关注基数、统计信息和锁等待。ANALYZE 会真实执行，写语句不能直接拿生产数据试。SQLite 侧可结合 EXPLAIN QUERY PLAN、事务时长和 busy 指标定位。

依据 / 对照：backend/app/infrastructure/database.py；backend/app/infrastructure/budget_connection.py

29. 【口述算法题】 怎样实现固定容量的 LRU Cache？

答：哈希表定位节点，双向链表维护访问顺序；get/put 命中移到最近端，超容量从最旧端删除并同步哈希表。均摊 O(1)，空间 O(capacity)。附录用 OrderedDict 演示，并处理 capacity=0。

30. 【口述系统设计题】 如果自己实现线程池，需要哪些核心组件，执行流程是什么？

答：线程池核心是有界队列、固定/受限 Worker、提交 Future、拒绝策略、异常回传和关闭流程。Worker 启动后循环阻塞取任务，由入队唤醒条件变量/队列等待者，执行完成再取下一项；不是每个任务重新建线程。需要定义队列满、取消、超时和 shutdown 的行为。工程后台 runner 用独立循环领取持久化任务，不等于通用线程池实现。

依据 / 对照：backend/app/jobs/runner.py；后半为基础设计

31. 追问：任务进入队列之后，由谁驱动 Worker 取出并执行？

答：线程池核心是有界队列、固定/受限 Worker、提交 Future、拒绝策略、异常回传和关闭流程。Worker 启动后循环阻塞取任务，由入队唤醒条件变量/队列等待者，执行完成再取下一项；不是每个任务重新建线程。需要定义队列满、取消、超时和 shutdown 的行为。工程后台 runner 用独立循环领取持久化任务，不等于通用线程池实现。

依据 / 对照：backend/app/jobs/runner.py；后半为基础设计

32. 如果项目涉及 Ray，会用它承担哪些工作？

答：项目未使用 Ray。若有大量独立 CPU/数据处理任务，可把计算交给 Ray 等执行层，但业务任务 ID、输入、结果与重试状态应存外部持久系统。Head 故障能否恢复要按具体组件及部署版本验证，不能假设所有内存任务都自动存活；重建调度与对象/Actor 状态是不同问题，副作用还需要幂等。

33. Ray Head 与 Worker 如何解耦？

答：项目未使用 Ray。若有大量独立 CPU/数据处理任务，可把计算交给 Ray 等执行层，但业务任务 ID、输入、结果与重试状态应存外部持久系统。Head 故障能否恢复要按具体组件及部署版本验证，不能假设所有内存任务都自动存活；重建调度与对象/Actor 状态是不同问题，副作用还需要幂等。

34. Head 故障后，任务调度、结果汇聚和后续任务怎样继续？

答：项目未使用 Ray。若有大量独立 CPU/数据处理任务，可把计算交给 Ray 等执行层，但业务任务 ID、输入、结果与重试状态应存外部持久系统。Head 故障能否恢复要按具体组件及部署版本验证，不能假设所有内存任务都自动存活；重建调度与对象/Actor 状态是不同问题，副作用还需要幂等。

九、百度 · 二面

1. 研发经历是否都与 Agent 有关？如果涉及方向转型，是在什么阶段？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

2. AI 应用平台具体是什么，客户是谁？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

3. 部署在哪里，目前规模多大？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

4. 你在平台中具体承担哪些工作？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

5. 为什么选择 LangGraph，它在系统中负责什么？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

6. 工具分层最终解决谁的问题？模型看到的不都是工具吗？

答：本项目没有开放式 Tool Calling 注册中心。模型输出固定 Pydantic 结构，程序只接受批准的动作和视图，再由本地函数执行。若扩展工具，注册表应绑定 name/version、输入输出 schema、执行器、权限、超时、幂等策略和副作用等级；模型看到的描述与执行器必须来自同一注册条目，调用前再校验参数与权限，不能只靠描述约束。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

7. 用户创建的 AI 应用如何评价，有哪些具体指标和数据？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

8. P95 TTFT 统计什么阶段？首响应较长时应如何分析原因？

答：项目没有流式 TTFT 实测。设计上客户端首个有意义内容到达时间与上游模型首 token 时间要分开计；P95 是同口径样本延迟的第 95 百分位。首响应慢先拆排队、鉴权/读取、安全分类、模型连接/推理及传输，再看锁等待、SQLite busy、上游限流和资源饱和。不能把 UI loading 或空心跳算作首个有效 token。

依据 / 对照：backend/app/observability/；后半为指标设计

9. 画布平台、Agent Harness 和客户交付是什么关系？

答：框架提供状态图和中断等原语，Harness 把模型调用、预算、授权、工具边界、业务完成条件、恢复和可观测性组合成实际运行环境。本项目在 LangGraph 之上补了 ModelBoundary、执行额度、回执、隐私血缘、输出策略与任务队列，属于专用业务 Harness，尚非通用插件运行平台。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/workflows/runtime.py；backend/app/policy/output.py

10. 客户最终购买什么，拿到什么交付物？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

11. 客户数据在这条链路中如何处理？

答：模型不能提交任意 SQL、字段或记录 ID；本地先绑定目标、批准固定视图，再用 ModelBoundary 发一次性授权，消费前累计计数。processing_plan 仅接收当前输入和操作状态，后续任务按需读取最少数据；local_only 禁止全部外发且随工作流恢复保留。当前没有多用户身份体系，不能把数据最小化等同于多租户鉴权。

依据 / 对照：backend/app/modeling/boundary.py；backend/app/modeling/payloads.py；docs/模型数据访问边界.md

12. 平台的执行引擎与独立 Agent 运行内核，本质区别是什么？

答：图引擎处理节点与边，Harness 还负责模型/工具授权、预算、业务完成条件、幂等、副作用、上下文和观测。本项目 LangGraph 是执行原语，Engine 加 Repository/Boundary/Policy 组合成专用 Harness；两者属于组合关系，不是两套独立 REST 服务。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

13. Skill 有什么作用？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

14. 追问：为什么还要把 Workflow 包装成 Skill？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

15. 企业 Agent 为什么不像 Codex、Claude Code 那样保持更开放的自主性？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

16. App 关闭再打开后，怎样继续看到原任务的流式输出？

答：当前不是 SSE token 流，也没有 Redis 会话状态；页面恢复业务状态不等于恢复流式文本。若扩展，用持久事件日志保存 task_id、单调 seq/event_id、类型及版本，重连携 Last-Event-ID，只补游标后的事件，再衔接实时流；客户端按 ID 去重。快照与增量要有一致边界，历史过期时显式要求重同步，不能仅靠 Redis TTL。

依据 / 对照：当前未实现；参照 backend/app/workflows/runtime.py 的业务状态恢复

17. 追问：SSE 重连怎样做到不漏字、不重复，协议具体怎么设计？

答：当前不是 SSE token 流，也没有 Redis 会话状态；页面恢复业务状态不等于恢复流式文本。若扩展，用持久事件日志保存 task_id、单调 seq/event_id、类型及版本，重连携 Last-Event-ID，只补游标后的事件，再衔接实时流；客户端按 ID 去重。快照与增量要有一致边界，历史过期时显式要求重同步，不能仅靠 Redis TTL。

依据 / 对照：当前未实现；参照 backend/app/workflows/runtime.py 的业务状态恢复

18. Workflow-as-Skill 中是否包含外部 I/O？

答：可以包含，但必须把外部操作 ID、幂等键、超时与重试语义纳入持久状态。当前 Workflow 中包含模型网络 I/O，ModelBoundary 控制发送；没有一般化 Workflow-as-Skill，也没有任意外部业务写入工具。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

19. 追问：这样一个 Workflow 与普通 API 服务有什么区别？

答：普通 API 常被视为一次请求/响应操作；Workflow Skill 表达可跨步骤、中断、人工输入和重试的持久任务，仍可能调用普通 API。差异在持久生命周期和恢复契约，而不是有没有 I/O。本项目的记录追问跨多次请求恢复，但未提供通用 Skill 协议。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

20. Human-in-the-loop 在哪里中断，如何恢复？

答：记录缺必填字段时 interrupt；计划首次或单次/累计超过阈值时等待确认。恢复请求必须匹配 thread_id、interrupt_id、request_id 与指纹，防止旧回答消费新中断。补充只改待补字段，保留日期与已知内容；计划确认还检查资料、基线与限制是否变化。拒绝、取消、过期都是独立状态，不能当作执行成功。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/planning.py；backend/tests/test_draft_clarification.py

21. Redis 中的状态过期以后，任务还能继续吗？

答：当前状态在两份 SQLite，未用 Redis。若未来 Redis 只做缓存，过期后从持久 workflow/checkpoint 重建；若唯一任务状态只存在 TTL 缓存，过期就无法可靠恢复。等待点、外部操作 ID 和结果应放持久存储，并明确业务有效期不能被缓存重建绕过。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

22. 上下文工程具体做了哪些事情？

答：工程优先不发送无关内容：首轮不带最近聊天和健康历史，后续只发经批准的固定视图。咨询饮食汇总最多近 7 天，体重/分量上下文最多近 28 天；明确更短范围就按更短范围。关键日期、数值、来源、未知状态和 local_only 必须保留。当前有字节和调用预算，未实现按 tokenizer 的动态 Token 分配或模型摘要压缩。

依据 / 对照：docs/模型数据访问边界.md；backend/app/modeling/boundary.py；backend/app/modeling/payloads.py

23. 项目是否包含 Memory，什么应该记、什么不应该记？

答：工作记忆保存待补草稿、中断和当前计划；事件账本保存健康事实及修订；结构化长期事实保存明确偏好、过敏和限制；派生结果保存规律、复盘和摘要。前者用 checkpoint，后三者主要在业务 SQLite。长期信息由明确陈述和本地校验写入，不把一次偏咸反馈自动变成长期偏好，也未使用向量库存记忆。

依据 / 对照：docs/四层记忆实现说明.md；backend/app/health/facts.py；backend/app/workflows/state.py

24. “支持同步、异步”具体指什么？

答：async 协程适合可等待 I/O，不会自动把同步阻塞函数变成非阻塞。同步端点可在线程池执行，CPU 密集任务考虑进程或独立任务服务；普通带 GIL 的 CPython 线程不能让纯 Python CPU 代码多核并行，需区分 free-threaded 构建。当前 Engine 还有锁和 SQLite 单所有者约束，单凭 FastAPI/async 不能推算商业容量。

依据 / 对照：backend/app/main.py；backend/app/workflows/concurrency.py；Python 官方 asyncio 文档

25. 追问：是否存在两套不同的 ReAct Loop？

答：当前没有两套 ReAct Loop，甚至没有开放式 ReAct 主循环。同步 HTTP、后台 runner 和人工恢复是不同触发入口，复用固定图与执行校验。设计上异步任务应持久化后挂起并续跑同一状态机，而非复制一份逻辑近似但行为不同的循环。

依据 / 对照：backend/app/main.py；backend/app/workflows/concurrency.py；Python 官方 asyncio 文档

26. 现在是否还独立写代码，能否现场完成一道简单 Coding？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

九、百度 · 三面

1. 介绍一下当前工作，重点讲技术设计、遇到的问题和解决方式。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. Harness 怎么设计，运行过程是什么，如何提升效果？

答：小程序提交 request_id 和处理模式；服务端检查回执与指纹，定位或创建 thread；安全入口先判定允许能力；模型解析草稿，本地校验缺失字段与权限；营养准备后保存完整独立记录，缺项则 interrupt 追问；最终本地渲染结果并保存回执。恢复时校验 thread、interrupt_id 和请求指纹，再继续原图。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py

3. 通用运行内核、业务扩展和服务入口怎样分工？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

4. 实际交付给客户的是中间件，还是云上的产品？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

5. ReAct、Skill 和 Browser Use 等能力运行在哪里？

答：当前只执行服务端固定 Python 工作流和模型请求，没有 Browser Use、通用 Skill 执行器或开放 ReAct。若新增浏览器/代码能力，应放独立受控执行环境，Engine 只持有授权与结果引用，不能给它宿主机任意权限。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

6. 为什么多个 Agent 共享 Worker？

答：工程没有主/子 Agent 或 Manager-Worker。若多个子任务可独立求解、需要不同权限或上下文，才值得引入；管理者分配目标/预算，Worker 只接收必要输入并返回带证据的结构结果。共享写入宜由单一汇聚者合并，或用版本 CAS；总预算、委派深度、去重 task_id 和取消传播防止互相转交形成循环。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

7. 不同用户和业务数据怎样隔离？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

8. 是否提供代码和 Bash 能力，代码在哪里执行？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

9. 为什么使用 LangGraph，而不是直接实现一个 Loop？

答：工程需要在追问、计划确认和重启后继续执行，LangGraph 的显式状态图、条件边、interrupt 与 SQLite checkpoint 正好覆盖这些需求。业务写入、请求幂等、外发权限和删除恢复仍由工程自行实现。相比直接写循环，减少了中断状态管理代码；代价是要理解节点重放及检查点语义，不能把持久化等同于外部操作只执行一次。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

10. 运行图包含哪些节点？

答：聊天图先 safety_gate，必要时 classify_safety；通过后 parse_initial/patch_pending，再 normalize_extraction、validate_drafts、authorize_operation、prepare_nutrition、save_ready；随后追问、persist、fact 或 respond，追问恢复重新过安全入口。具体边与节点见 backend/app/workflows/chat_flow.py 的 build_chat。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py

11. 追问：LangGraph 能否替换，替换会影响哪些部分？

答：可以替换，但需重新实现状态转移、持久化中断、检查点加载/重放及恢复语义，并迁移旧 checkpoint。业务规则、Repository 和模型授权可尽量保留；直接把 graph.invoke 换成 while 并不能保留现有保证。应先用恢复、删除并发和幂等回归证明等价。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

12. Skill 中的代码怎样运行？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

13. Workflow 形式的 Skill 如何定义和执行？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

14. LangGraph Checkpoint 与自行建设的持久化恢复机制，有什么区别？

答：LangGraph 用 thread_id 从 checkpoints.db 找图状态与待恢复位置，业务库保存 workflow、请求回执和恢复尝试。保存的不只是消息，还包括草稿、待补字段、已写记录索引及处理模式。重启恢复后仍要重新验证权限、数据版本和期限；两份 SQLite 不是跨库原子事务，靠幂等键、回执和恢复流程处理窗口。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/state.py；backend/tests/test_resume_recovery.py

15. 如何评估性能和稳定性？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

16. 如果一次工具调用失败，但最后任务成功了，成功率应该怎么算？

答：区分请求/工作流终态、工具尝试成功率和任务验收率：一次工具失败后任务达成，任务成功但尝试失败仍计入工具分母。工程有工作流/节点/模型调用与任务运行指标，业务质量账本记录营养未知、必需追问、纠正和计划结果。不能把 HTTP 200、模型返回文本或风险被拦截一律当成普通任务完成。

依据 / 对照：backend/app/observability/；backend/app/infrastructure/repositories/observability.py

17. Agent 的 Ground Truth 和测试用例如何构建？

答：以合成输入和隔离临时库构造正常、缺失、纠错、重试、重启、删除并发和恶意输出用例，断言最终事实、状态与必要轨迹。Ground Truth 应允许等价执行路径，但禁止跳过安全/审批。LLM Judge 可辅助表达评价，关键数值、保存状态与权限用确定性断言；同类模型自评会有偏差，要人工复核、留出测试集并观察分组结果。

依据 / 对照：backend/tests/test_grounding.py；backend/tests/test_output_policy.py；backend/tests/test_safety_routing.py

18. 追问：怎样判断任务真正完成？

答：任务成功应由本地状态和交付物判定：记录已写入且 schema 合法；计划已满足确认条件且基线未变；删除作业完成且旧请求无法复活。图边与写入前检查强制必要步骤，模型文字“已完成”不能作凭据。缺产物时返回失败/待补充，可做有限补救并保留失败原因；不把检查没通过的结果记作成功。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/planning.py；backend/app/workflows/concurrency.py

19. 耗时、性能和 Token 消耗目前是什么水平？

答：可引用仓库已有报告而非本次实测：2026-10-27 发布阶段合成本地体重工作流，关闭/开启观测的 P95 为 83.77/87.83 ms，约增加 4.85%；按每模式 200 个样本、交替轮次的报告口径，模型网络与调度关闭。这只能说明该环境的观测开销，不能作线上端到端延迟、TTFT、QPS 或容量承诺；也没有排障效率提升的对照数据。

依据 / 对照：docs/evidence/observability-validation-summary.json

20. 追问：如何判断首个可见响应时间是否过长，统计的究竟是哪段时间？

答：项目没有流式 TTFT 实测。设计上客户端首个有意义内容到达时间与上游模型首 token 时间要分开计；P95 是同口径样本延迟的第 95 百分位。首响应慢先拆排队、鉴权/读取、安全分类、模型连接/推理及传输，再看锁等待、SQLite busy、上游限流和资源饱和。不能把 UI loading 或空心跳算作首个有效 token。

依据 / 对照：backend/app/observability/；后半为指标设计

21. 实际客户、终端用户和并发规模如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

22. BM25、Embedding、RRF、Reranker 与 MMR 怎么组合？

答：BM25 补关键词精确匹配，向量补语义召回；分别取候选，RRF 按名次融合而非直接相加不同量纲分数，再去重、用 Reranker 排相关性。MMR 在相关性与结果多样性间权衡，适合候选重复时选用；若降低证据完整性或延迟不值就不启用。本项目未使用这些组件，具体收益必须由固定数据集消融验证。

依据 / 对照：官方原理参考：Elastic RRF；本项目未实现

23. 如果要继续提高召回率，最难的部分是什么？

答：当前没有 RAG 实测。设计上最难的往往不是继续加候选，而是补齐真实问题与可复核证据标注，区分语料缺失、解析丢失、过滤错误、切分关系断裂和排序误差；盲目增加 K 会扩大噪声与成本。先对失败切片逐阶段定位，再做受控消融。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

24. RAG 评测是否亲自做过，用过哪些指标？

答：Recall@K 看已标注相关证据召回覆盖，MRR 看第一个相关结果的位置，NDCG 看带相关等级的排序质量；多证据问答还应检查证据集合是否齐全。回答侧单独测正确性、依据一致性、拒答和引用定位，联合记录延迟/成本。项目尚无 RAG 指标实测，不能引用安全分类测试替代检索评测。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

25. Recall@K、MRR、NDCG 分别适合衡量什么？

答：Recall@K 看已标注相关证据召回覆盖，MRR 看第一个相关结果的位置，NDCG 看带相关等级的排序质量；多证据问答还应检查证据集合是否齐全。回答侧单独测正确性、依据一致性、拒答和引用定位，联合记录延迟/成本。项目尚无 RAG 指标实测，不能引用安全分类测试替代检索评测。

依据 / 对照：扩展评测设计；项目没有 RAG 基准

26. 是否关注 Codex、Claude Code、WorkBuddy 等新方案？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

27. 是否阅读源码，最近看到了哪些设计变化？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

28. 你提到的新 Harness 有什么优势？这种优势是绝对的吗？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

29. 为什么现在考虑换工作？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

十、深信服 · 一面

1. 请做一下自我介绍。

答：可这样介绍项目：轻记是个人健康记录 Agent，用自然语言完成饮食、体重、睡眠、运动和生理周期记录，并支持纠错、查询、计划确认及复盘。前端是 Taro 微信小程序，服务端是 FastAPI，LangGraph 管理有状态流程，SQLite 保存业务和检查点。重点可讲持久化追问、最小数据外发和删除后防止旧任务写回。个人职责、任职时间需按真实经历补充。

依据 / 对照：README.md；backend/app/workflows/engine.py

2. 最近的工作主要都与 Agent 有关吗？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

3. 具体涉及平台研发、框架开发还是业务应用落地？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

4. 选一个项目，讲讲怎样快速落地 Agent 应用。

答：可先用“中午吃了牛肉面—分量缺失追问—补充后保存—不是大碗是小碗纠错”跑通最小闭环。实现顺序是契约与本地规则、固定图、幂等/持久化、模型理解、边界校验、合成验收，最后接小程序；先证明闭环可恢复，再增加计划与记忆。这里描述工程可演示流程，不虚构个人交付周期。

依据 / 对照：backend/app/workflows/runtime.py；backend/app/workflows/chat_flow.py

5. 刚才业务场景讲得比较多，能否进一步介绍 Agent 架构设计？

答：入口层 api/ 处理 HTTP 契约和依赖；workflows/ 组织状态、路由、中断和恢复；health/、chat/、policy/ 承载健康业务与回复规则；modeling/ 做模型适配和数据边界；infrastructure/ 负责持久化。它们装配在一个服务内，尚未抽成独立微服务。可复用执行机制，但健康规则仍和当前 Engine 有耦合。

依据 / 对照：backend/app/main.py；backend/app/workflows/engine.py

6. 框架更偏固定 Workflow，还是自主规划 Agent？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

7. 在 Agent 的具体设计中，你承担了哪些工作？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

8. 参考通用 Coding Agent 的实现思路，当前业务 Agent 还有哪些可以改进的地方？

答：优先补经核验营养来源、真实设备验收和更大独立评测，再做鉴权与租户隔离、独立 Engine 服务及容量验证。只有出现明确任务需求才引入 RAG、多 Agent 或动态 DSL。探索任务可放宽规划自由，健康写入/计划激活仍要受本地规则约束。迁移其他行业应替换事实模型和领域策略，不能承诺当前健康规则直接通用。

依据 / 对照：README.md；docs/调度器可靠性与独立Worker开发方案.md

9. 如何将具体业务场景和固定 Workflow 加入 ReAct 执行过程？

答：当前采用模型提出结构化处理计划、本地程序审核并沿固定图执行。记录校验、营养准备、保存、计划确认属于确定性步骤，不能让模型跳过。探索性任务可设计 ReAct 自主选工具，但本项目没有开放 ReAct 主循环。复杂度本身不足以证明需要多 Agent，应先检查是否需要独立上下文、权限或真正可并行的子目标。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

10. 追问：有哪些具体参考例子？

答：代码与测试体现的一个问题是：“帮我记录昨天吃得有点咸”缺少具体食物，不能写空餐食，也不能在后续补充时丢掉原日期。方案是先提取允许不完整的草稿，本地判断必填字段，interrupt 持久化待补项，恢复仅 patch 当前字段，并用日期/证据/重放测试验收。这是可从仓库复现的技术案例，不据此虚构个人线上故障或收益。

依据 / 对照：backend/tests/test_draft_clarification.py；backend/tests/test_contextual_replies.py；backend/app/workflows/chat_flow.py

11. 是否有后端或平台研发经验？如有，涉及哪些方面？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

12. 是否有 NLP 相关研究经历？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

13. 工作中是否参与过大模型训练？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

14. 当前 Agent 项目涉及模型训练，还是主要调用模型 API 或本地部署模型？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

十一、同花顺 · 一面

1. 近期的主力开发语言是什么？如使用过 Python 或 Java，分别用于哪些场景？

答：本项目主要调用兼容模型 API，未见预训练、微调、后训练或自部署模型推理代码。改进主要来自结构化解析、上下文最小化、本地规则、输出验证和评测。Python 用于后端与测试，TypeScript/React/Taro 用于小程序；仓库不能证明 Java/NLP 研究经验，个人经历应另据实说明。

依据 / 对照：backend/requirements.txt；miniprogram/package.json；backend/app/modeling/

2. 公司内部的技术栈是否受到合规等限制？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

3. 团队使用哪些 AI Coding 工具和模型？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

4. AI 应用平台是什么产品，解决什么问题？

答：本项目面向个人健康记录，是单用户 ToC 开发原型。交付是微信小程序代码、Python API、运行配置和使用文档；支持自然语言记录、历史、计划、发现和数据管理。没有仓库证据支持商业客户数量、付费转化或减重效果，不能包装成企业级低代码平台或已规模商业化产品。

依据 / 对照：README.md；miniprogram/README.md

5. 已有很多低代码 AI 平台，为什么仍然选择自研？

答：本项目针对结构化健康事实、安全边界、追问恢复和精确删除定制；并未实现 Dify/Coze 式画布或开放 Coding Agent，也无与它们同模型同任务的 A/B 数据。比较应固定模型、工具、数据、权限、预算和任务，测完成率、越权率、延迟/成本及维护投入。只能论证当前约束为何需要这些机制，不能声称全面优于某框架或产品。

依据 / 对照：README.md；backend/tests/；比较方案未执行

6. 平台是否上线，用户量、调用量和并发量如何？

答：当前每个数据目录只允许一个 Engine，默认同时活跃工作流上限为 2，后台任务并发为 1。这些是配置和实现约束，不是压测容量；仓库没有生产用户量、QPS 或完整并发容量结论。回答时应分别给部署实例数、限制、压测环境与真实流量，不能相互替代。

依据 / 对照：backend/app/common/config.py；backend/app/infrastructure/data_owner.py；backend/app/jobs/runner.py

7. LangGraph、FastAPI 和 Python Worker 为什么能够支撑商业化平台？

答：当前小程序独立构建发布，FastAPI 只提供 API；本地 Uvicorn 或 Linux Compose 加固方式运行。一个 Engine 装配聊天、分析和计划图及后台任务线程，逻辑图不是独立进程。不能为同一目录加多个 Uvicorn worker。私有环境可接内网兼容模型，或服务端强制 local_only，但后者只保留受限本地能力，不等于完整离线语义模型。

依据 / 对照：README.md；Dockerfile；compose.hardened.yaml；backend/app/main.py

8. 如果 Workflow 没有 LLM 节点，而是大量代码节点，应该怎样执行？

答：没有 LLM 节点就按普通受控 DAG/Workflow 执行，参数与依赖静态校验，I/O 与 CPU 任务分配不同执行池。用户可编辑代码必须进隔离执行进程，硬超时强杀；可信内置函数也需预算与故障状态。本工程只有服务端维护的固定 Python 节点，没有通用用户代码节点。

依据 / 对照：backend/app/workflows/chat_flow.py；backend/app/workflows/chat_parsing.py

9. 追问：代码中出现死循环怎么办？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

10. ReAct 的核心执行逻辑是什么？

答：ReAct 若反复选相同工具、拿不到成功信号、失败无限重试，或压缩丢失目标就会不收敛。设计上要同时限制步骤、模型调用、总时间和输出大小，并检测无进展。工程当前是固定图，默认执行期限 120 秒、模型超时 45 秒、安全分类 8 秒，且有累计调用/字节额度；可选信息最多追问两轮，必填缺失不能为了结束而保存空记录。

依据 / 对照：backend/app/common/config.py；backend/app/common/execution_budget.py；backend/app/workflows/runtime.py

11. 以查询天气、控制电视和空调为例，从用户输入到 Function Call、Observation 和 Final Answer，完整链路怎样运行？

答：该场景未实现。设计链路是解析意图，工具注册表匹配天气/设备动作，参数和用户设备权限校验，再调用工具；Observation 返回天气或实际设备状态，最后根据结果回复。控制设备属于写操作，需按风险确认、幂等键和回读状态，不能用模型的 Final Answer 证明开关成功。

依据 / 对照：backend/app/modeling/payloads.py；backend/app/modeling/boundary.py

12. 工具数量增加后，怎样避免选错工具或参数错误？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

13. 渐进式披露具体怎样实现？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

14. 几十个 Skill 如何被发现和激活？

答：当前能力固定，没有几十个 Skill 的动态路由器。扩展时先按用户权限和业务域过滤，再给模型简短能力摘要，选中后才加载详细 schema 和资源；每轮按 capability_id/version 去重，不把全文反复追加。路由评测需看正确选择、拒答/澄清、权限越界和最终任务完成，不能只看名称命中率；执行层仍须再鉴权。

依据 / 对照：参照 backend/app/modeling/payloads.py；扩展设计

15. Skill 同时包含 Markdown、脚本、代码和工具时，运行时怎样执行？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

16. Workflow-as-Skill 如何与 ReAct 主循环衔接？

答：项目未实现通用 Skill 加载/发现协议。概念上 Tool 是单个可校验动作；Skill 描述完成一类任务的规则、资源和能力组合；Workflow 是可执行的状态转移。若将本项目记录图包装为 Skill，需要固定输入输出、版本、权限、超时和等待协议；运行时调用图后保存 continuation，完成后把结构化结果交还主循环，而不是直接执行一份 Markdown。

依据 / 对照：backend/app/workflows/chat_flow.py；此 Skill 包装未实现

17. 为什么有了 MCP 之后，还需要 Skill？

答：本工程未接 MCP。MCP 提供标准化能力协商、工具发现及调用契约，Tool 是动作，Skill 是任务方法；Skill 可以调用 MCP，也可直接调用受控 API/CLI。MCP 不会自动保证租户隔离、审批和安全，这些治理也不是协议独有。是否接入取决于多客户端复用、发现和维护成本，不能说有 MCP 就不需要 Skill 或 API。

依据 / 对照：MCP Tools 官方规范；项目未集成

18. 反过来，有了 Skill，MCP 能不能完全不用？

答：本工程未接 MCP。MCP 提供标准化能力协商、工具发现及调用契约，Tool 是动作，Skill 是任务方法；Skill 可以调用 MCP，也可直接调用受控 API/CLI。MCP 不会自动保证租户隔离、审批和安全，这些治理也不是协议独有。是否接入取决于多客户端复用、发现和维护成本，不能说有 MCP 就不需要 Skill 或 API。

依据 / 对照：MCP Tools 官方规范；项目未集成

19. Skill 直接调用 OpenAPI、CLI 或临时生成代码，为什么不能替代 MCP？

答：本工程未接 MCP。MCP 提供标准化能力协商、工具发现及调用契约，Tool 是动作，Skill 是任务方法；Skill 可以调用 MCP，也可直接调用受控 API/CLI。MCP 不会自动保证租户隔离、审批和安全，这些治理也不是协议独有。是否接入取决于多客户端复用、发现和维护成本，不能说有 MCP 就不需要 Skill 或 API。

依据 / 对照：MCP Tools 官方规范；项目未集成

20. 追问：如果 API Key 已在环境变量中，模型只需要执行简化命令，MCP 还提供什么额外价值？

答：本工程未接 MCP。MCP 提供标准化能力协商、工具发现及调用契约，Tool 是动作，Skill 是任务方法；Skill 可以调用 MCP，也可直接调用受控 API/CLI。MCP 不会自动保证租户隔离、审批和安全，这些治理也不是协议独有。是否接入取决于多客户端复用、发现和维护成本，不能说有 MCP 就不需要 Skill 或 API。

依据 / 对照：MCP Tools 官方规范；项目未集成

21. MCP 相比标准 OpenAPI 真正增加了什么？

答：本工程未接 MCP。MCP 提供标准化能力协商、工具发现及调用契约，Tool 是动作，Skill 是任务方法；Skill 可以调用 MCP，也可直接调用受控 API/CLI。MCP 不会自动保证租户隔离、审批和安全，这些治理也不是协议独有。是否接入取决于多客户端复用、发现和维护成本，不能说有 MCP 就不需要 Skill 或 API。

依据 / 对照：MCP Tools 官方规范；项目未集成

22. 你提到的安全、治理和复用价值，是 MCP 独有的吗？

答：本工程未接 MCP。MCP 提供标准化能力协商、工具发现及调用契约，Tool 是动作，Skill 是任务方法；Skill 可以调用 MCP，也可直接调用受控 API/CLI。MCP 不会自动保证租户隔离、审批和安全，这些治理也不是协议独有。是否接入取决于多客户端复用、发现和维护成本，不能说有 MCP 就不需要 Skill 或 API。

依据 / 对照：MCP Tools 官方规范；项目未集成

23. 面向千万用户的云端 Agent，整体怎样设计？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

24. 是否需要给每个用户运行一个独立 Agent 实例？

答：不需要每用户独占常驻 Agent 进程。逻辑状态按 user/task 持久化，计算由受配额控制的 Worker 池调度，只有需要强隔离的不可信执行再分配沙箱。本项目仍限制每数据目录一个 Engine，不能直接通过增加 Uvicorn worker 实现这种云端架构。

依据 / 对照：当前无实现；可借鉴 backend/app/workflows/concurrency.py 的版本检查

25. 用户自定义 Skill、长期记忆、偏好和上传文件如何加载、隔离？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

26. 公共资产与租户私有资产如何做到物理复用和逻辑隔离？

答：当前只有单用户、单数据目录，不具备多租户隔离。扩展时应在认证主体、数据库查询、缓存键、对象存储、任务队列和外部凭据处统一绑定 tenant/user，拒绝客户端自行声明身份；公共只读资产可按内容复用，私有状态必须隔离。逻辑 Agent 不必独占进程，但执行不可信代码可按风险分配独立沙箱。

依据 / 对照：README.md；backend/app/infrastructure/data_owner.py；以下为扩展设计

27. 用户要求执行 ps -ef 或终止某个 Java 进程时，如何保证安全？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

28. Agent 在沙箱内运行与在沙箱外调度，架构有什么不同？

答：调度器在沙箱外时可集中管理权限、状态与预算，沙箱只执行受限动作；全部放沙箱内更易封闭执行依赖，但调度恢复、凭据和控制通道仍要设计。本工程是外部服务执行固定图，没有用户代码沙箱。应以宿主机资源不可触达、网络最小出口和可强制终止为验收条件。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

29. 云端 Agent 的性能、资源成本、隔离和安全如何权衡？

答：开发模式没有 OS 级沙箱，也不执行用户任意 Bash/代码。加固 Compose 提供非 root、只读代码、独立数据卷、临时目录配额、资源限制及受控 HTTPS 出口代理。隔离收益要靠实际探测验证，配置存在不等于验收通过。若开放不可信代码，需独立执行容器/微虚机、超时强杀、文件网络白名单和无宿主机密钥环境。

依据 / 对照：compose.hardened.yaml；deploy/network-policy.md；docs/运行环境最小权限修复开发方案.md

30. 【Python 编码题】 编写一个协程示例。

答：用 async def 定义协程，await 等待非阻塞 I/O，asyncio.gather 并发组织独立任务，asyncio.run 启动入口；只调用协程函数不会自动执行。附录演示两个异步任务和结果汇聚，避免在协程中使用 time.sleep。

31. 【Java 编码题】 给出两种单例实现方案。

答：可用静态内部类 Holder 延迟初始化，或 volatile + 类级同步锁的双重检查实现。两者都要私有构造器；DCL 需两次检查并用 volatile 保证安全发布。附录提供 Java 代码，此题不属于当前 Python 工程已有能力。

32. 【Java 编码题·追问】 双重检查锁为什么需要 volatile，锁应该加在哪里？

答：先校验步骤类型、依赖和引用，再查询两个城市的天气，温差由本地代码计算，Summarizer 只读已确认结果。附录提供可运行模拟天气示例，明确 tool 与 compute 类型；不调用真实天气 API，也不声称本项目已有通用计划执行器。

33. 平时是否在本机使用 Codex 等 AI Coding 工具？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

34. 项目代码由谁编写，AI Coding 占多大比例？

答：代码无法证明个人职级、薪酬、晋升、离职原因、团队人数、工作地点、阅读经历或 AI 编码比例。这类问题请据实回答，按“背景—本人负责模块—关键决策—验证结果”组织；可用本项目展示技术理解，但不能把仓库内全部工作直接归为个人贡献。

附录 A · 编码练习

以下为独立练习，不是工程已有业务。Python 版本已运行内置边界检查；ReAct 为接口级伪代码，Java 版本为参考实现，未在本机编译。天气数据固定模拟，不调用外部服务。

Two Sum

期望时间 O(n)，空间 O(n)；先查后写，支持 [3,3]。

```
def two_sum(nums, target):
    seen = {}
    for i, value in enumerate(nums):
        if target - value in seen:
            return seen[target - value], i
        seen[value] = i
    return None
```

原地合并

时间 O(m+n)，额外空间 O(1)，第一个数组已有足够预留空间。

```
def merge_sorted(a, m, b, n):
    i, j, k = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and a[i] > b[j]:
            a[k] = a[i]
            i -= 1
        else:
            a[k] = b[j]
            j -= 1
        k -= 1
```

相邻元素不同

最大频次 <= ceil(n/2) 是可行条件；时间 O(n log k)，空间 O(k)。示例约定元素为可比较的整数。

```
def rearrange(nums):
    counts = Counter(nums)
    if max(counts.values(), default=0) > (len(nums) + 1) // 2:
        return None
    heap = [(-count, value) for value, count in counts.items()]
    heapify(heap)
    previous = (0, None)
    result = []
    while heap:
        count, value = heappop(heap)
        result.append(value)
        if previous[0] < 0:
            heappush(heap, previous)
        previous = (count + 1, value)
    return result
```

二叉树层序

时间 O(n)，辅助空间 O(w)，不含结果空间。

```
def level_order(root):
    if root is None:
        return []
    queue, result = deque([root]), []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
```

```
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
        result.append(level)
    return result
```

LRU

OrderedDict 实现期望 O(1) 访问/更新；面试要求手写时用哈希表 + 双向链表替换。

```
class LRU:
    def __init__(self, capacity):
        if capacity < 0:
            raise ValueError('negative capacity')
        self.capacity, self.data = capacity, OrderedDict()

    def get(self, key, default=None):
        if key not in self.data:
            return default
        self.data.move_to_end(key)
        return self.data[key]

    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        while len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

协程

两个独立异步等待并发执行；只有 await/create_task/gather 等驱动后才实际运行。

```
async def coroutine_demo():
    async def fetch(label, seconds):
        await asyncio.sleep(seconds)
        return label
    return await asyncio.gather(fetch('A', 0.01), fetch('B', 0.01))
```

Plan-and-Execute

Planner 返回固定类型计划，Executor 校验并解析前序结果引用，计算步骤本地完成。此示例按给定拓扑顺序执行；生产版本还需持久化、权限、预算与失败状态。

```
def weather_plan():
    # Fixed, validated example plan: no arbitrary expressions or eval.
    return [
        {'id': 'a', 'kind': 'tool', 'name': 'weather',
         'args': {'city': 'Shanghai'}},
        {'id': 'b', 'kind': 'tool', 'name': 'weather',
         'args': {'city': 'Beijing'}},
        {'id': 'delta', 'kind': 'compute', 'name': 'difference',
         'args': {'left': {'ref': 'a', 'field': 'temperature_c'},
                  'right': {'ref': 'b', 'field': 'temperature_c'}}},
    ]


def execute_weather(steps):
    # Mock temperature data for exercising execution, not real weather.
    weather = {'Shanghai': 24, 'Beijing': 18}
    results = {}
    def resolve(value):
        if not isinstance(value, dict):
            return value
        if set(value) != {'ref', 'field'}:
            raise ValueError('invalid reference')
        return results[value['ref']][value['field']]
    for step in steps:
        if (set(step) != {'id', 'kind', 'name', 'args'}
```

```
                or not isinstance(step['id'], str)
                or step['id'] in results):
            raise ValueError('invalid step')
        if (step['kind'], step['name']) == ('tool', 'weather'):
            if set(step['args']) != {'city'}:
                raise ValueError('invalid weather arguments')
            results[step['id']] = {
                'temperature_c': weather[step['args']['city']]}
        elif (step['kind'], step['name']) == ('compute', 'difference'):
            if set(step['args']) != {'left', 'right'}:
                raise ValueError('invalid difference arguments')
            left, right = map(resolve, (step['args']['left'],
                                        step['args']['right']))
            results[step['id']] = {'temperature_delta_c': left - right}
        else:
            raise ValueError('unsupported step')
    return results
```

ReAct Loop + Hook（伪代码）

内核必须强制授权、预算和完成校验；Hook 可提供建议/上下文，不能覆盖这些约束。外部副作用状态未知时需停下来对账，不能当成普通失败重复执行。

```
def react(task, runtime, hooks):
    state = runtime.load_or_create(task)
    for step in range(runtime.max_steps):
        runtime.check_deadline_and_cancel(state)
        context = runtime.project_authorized_context(state)
        context = hooks.before_model(context, state)
        proposal = runtime.model(context)
        proposal = runtime.validate_schema(proposal)
        if proposal.kind == "final":
            hooks.before_finish(state, proposal)
            runtime.verify_required_results(state)
            return runtime.commit_success(state, proposal)
        call = runtime.resolve_and_authorize(proposal, state)
        receipt = runtime.find_tool_receipt(call.idempotency_key)
        if receipt is None:
            receipt = runtime.execute_with_timeout(call)
            runtime.persist_tool_receipt(receipt)
        runtime.apply_observation(state, receipt)
        hooks.after_tool(state, receipt)
        runtime.check_progress_and_checkpoint(state)
    return runtime.commit_failure(state, "step_limit")
```

Java 单例：Holder 与 DCL

```
final class HolderSingleton {
    private HolderSingleton() {}
    private static class Holder {
        static final HolderSingleton INSTANCE = new HolderSingleton();
    }
    static HolderSingleton getInstance() {
        return Holder.INSTANCE;
    }
}

final class DclSingleton {
    private static volatile DclSingleton instance;
    private DclSingleton() {}
    static DclSingleton getInstance() {
        if (instance == null) {
            synchronized (DclSingleton.class) {
                if (instance == null) {
                    instance = new DclSingleton();
                }
            }
        }
        return instance;
    }
}
```

Holder 利用类初始化语义；DCL 的 volatile 配合类级锁实现安全发布。不能把 sleep(0) 当作可见性或同步手段。反射、序列化等额外约束需根据题面单独讨论。

验证范围

独立 Python 脚本检查：Two Sum 重复值/空输入；原地合并及空首数组；重排可行/无解；树空输入/单支；LRU 访问淘汰/零容量；协程汇聚；天气依赖引用与未知工具拒绝。这里只验证练习代码，不等于后端全套回归。

附录 B · 备考与证据索引

两分钟开场

用“自然语言记录如何减少手工录入”交代需求，随后讲“固定工作流约束模型”和“事实可纠错可删除”两项选择。个人职责只讲确实参与的内容；没有用户/收益数据时直接说明原型阶段。

优先练三条时序

记录缺分量：解析草稿→待补→中断→重启→绑定回答→准备营养→幂等保存。计划变更：生成草案→确认→资料/基线重新校验→生效或过期。删除：影响预览→作业持久化→清理业务/检查点→旧请求与晚到模型结果被拒绝。

性能与效果分开

历史性能报告是本地合成流程，不能说线上 P95。安全集只有 16 条合成样本，风险拦截成功也不代表路由完全准确。对照实验要固定模型、工具、数据和预算，保留失败切片和成本，不能只报成功案例。

原题中的未实现方向

RAG 练一条解析→召回→重排→证据回答的独立最小例子；多 Agent 练父子任务关联和结果汇聚；SSE 练事件 ID、补发和去重；MCP 练工具契约和执行鉴权。这些完成前继续标为设计讨论。

面试自检

每个设计连续回答：为什么需要、替代方案是什么、失败窗口在哪、数据在哪里、谁有执行权、如何验证、尚未保证什么。不要把一个锁说成幂等，不要把一次模型输出说成最终保存。

工程证据索引

执行与恢复

backend/app/workflows/engine.py；runtime.py；state.py；chat_flow.py；concurrency.py；planning.py

模型授权与输出

backend/app/modeling/boundary.py；payloads.py；transport.py；backend/app/policy/output.py

后台可靠性

backend/app/infrastructure/repositories/jobs.py；outbox.py；backend/app/jobs/runner.py；context.py

长期信息与删除

docs/四层记忆实现说明.md；docs/精准删除与数据血缘解决方案.md；docs/模型数据访问边界.md

历史验证报告

docs/evidence/observability-validation-summary.json；scheduler-validation-summary.json；safety-evaluation-2026-10-26.json

相关回归

backend/tests/test_resume_recovery.py；test_runtime_recovery.py；test_workflow_concurrency.py；test_structured_facts.py；test_privacy_deletion.py；test_safety_routing.py；test_output_policy.py

小程序与部署

miniprogram/package.json；miniprogram/README.md；README.md；compose.hardened.yaml；Dockerfile

核验用官方参考

LangGraph persistence：检查点与长期存储
https://docs.langchain.com/oss/python/langgraph/persistence

MCP tools：发现和调用契约
https://modelcontextprotocol.io/specification/2025-11-25/server/tools

Python asyncio：协程与任务
https://docs.python.org/3/library/asyncio-task.html

Java 线程与内存模型
https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html

Elastic RRF：按名次融合
https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion

外部资料用于核对通用原理，不作为本项目已集成对应组件的证据。工程事实以当前源码为准；README 中旧 schema 版本说明与后续段落并存，材料不将旧段落的版本号作为现状。